Bastille Documentation

Release 1.4.3.260429

Christer Edwards

Jun 06, 2026

CONTENTS:

Welcome to the official Bastille documentation. This collection of documents will outline installation and usage of Bastille.

While reading the documentation and using Bastille, you will find that sometimes “container” is used, and sometimes “jail” is used. These are completely interchangeable, but there is some debate as to which one is more correct. Be that as it may, anytime you read “container” or “jail”, it means a FreeBSD jail.

The latest version of this documentation can always be found at https://docs.bastillebsd.org.

CHAPTER

ONE

COMPARING

Most jail managers have a table showing what they and their competitors are capable of. While this is a good idea, the maintainers and developers of each jail manger do not regulary visit each others projects to update these tables.

Below is a table of what we feel is most important for a jail manager, as well as a list of popular managers and their status on each option.

Feature	BastilleBSD	Appjail	pot	ezjail	iocage
OCI Compliant	No	Yes	No	No	No
Writen In	Bourne Shell	Bourne Shell, C	Bourne Shell, Rust	Bourne Shell	Bourne Shell, Python
Dependencies	None	C	Rust	None	Python
Jail Types	vnet, bridged vnet, thin, thick, empty, clone, Linux	clone, copy, tiny, thin, thick, empty, linux+de bootstrap	thick	basejail	clone, basejail, template, empty, thick
Jail Dependency	Yes	Yes	Yes	No	Yes
Import/ Export	Yes	Yes	Yes	Yes	Yes
Boot Order Priorities	Yes	Yes	No	Yes using 'rcorder'	Yes
Linux Containers	Yes	Yes	No	No	Yes
Automation	Templates	Makejail, Initscripts, Images	Flavours, Images	Flavours	Plugins
Cloning	Yes	No	No	No	No
Package Management	Yes	No	No	No	No
ZFS Support	Yes	Yes	Yes	No	Yes
Volume Management	Basic	Yes	Basic	No	Basic
VNET Support	Yes	Yes	Yes	No	Yes
IPv6 Support	Yes	Yes	Yes	Yes	Yes
Dual Network Stack	Yes	Yes	Yes	No	No
Netgraph	Yes	Yes	No	No	No
Dynamic Firewall	Yes	Yes	Yes	No	No
Dynamic DEVFS Ruleset Management	No	Yes	No	No	No
Resource Control	Yes	Yes	CPU and Memory	No	Legacy Only
CPU Sets	Yes	Yes	Yes	Yes	Yes
Parallel Startup	Yes	Yes (Health checkers, jails & NAT)	No	No	No
PkgBase Support	Yes	Yes	No	No	No
Multi-target Commands	Yes	No	No	No	No
Log Management	Basic (console logs)	Yes	No	No	No
Copy Files Between Jails	Yes	No	No	No	No
Automated Jail Migration Between Servers	Yes	No	No	No	No
Top/Htop Support	Yes	No	No	No	No

We do our best to stay true and honest as to what other jail managers do and don't do. If you see an error, you can open a PR on the BastillBSD github repo.

We also realize that each jail manger does certain things better than other, and perhaps certain things worse. Some do this, others do that. They are all different, and each user should choose the one they want to use based on their needs.

Thanks for using BastilleBSD!

CHAPTER

TWO

INSTALLATION

Bastille is available in the official FreeBSD ports tree at `sysutils/bastille`. Binary packages are available in quarterly and latest repositories.

Current version is `1.4.3.260429`.

To install from the FreeBSD package repository:

- quarterly repository may be older version
- latest repository will match recent ports

2.1 pkg

```
pkg install bastille
```

2.2 ports

```
make -C /usr/ports/sysutils/bastille install clean
```

2.3 git

```
git clone https://github.com/BastilleBSD/bastille.git
cd bastille
make install
```

The `git` method will install the latest files from GitHub directly onto your system. It is verbose about the files it installs (for later removal), and also has a `make uninstall` target. You may need to manually copy the sample config into place before Bastille will run. (ie; `/usr/local/etc/bastille/bastille.conf.sample`)

Note: installing using this method overwrites the version variable to match that of the source revision commit hash.

CHAPTER

THREE

GETTING STARTED

Bastille has many different options when it comes to creating and managing jails. This guide is meant to show some basic setup and configuration options.

3.1 Setup

The first command a new user should run is `bastille setup`. This will configure the networking, storage, and firewall on your system for use with Bastille.

By default the `bastille setup` will configure a loopback interface, storage (ZFS if enabled, otherwise UFS) and the `pf` firewall.

Alternatively, you can run `bastille setup OPTION` command with any of the supported options to configure the selected option by itself.

To see a list of available options, see the *setup* subcommand.

```
ishmael ~ # bastille setup
```

Now we are ready to bootstrap a release and start creating jails.

3.2 Bootstrapping a Release

To bootstrap a release, run `bastille bootstrap RELEASE`.

```
ishmael ~ # bastille bootstrap 14.2-RELEASE
```

This will fetch the necessary components of the specified release, and enable us to create jails from the downloaded release.

3.3 Creating a Jail

There are a few different types of jails we can create, described below.

- Thin jails are the default, and are called thin because they use symlinks to the bootstrapped release. They are lightweight and are created quickly.
- Thick jails use the entire release, which is copied into the jail. The jail then acts like a full BSD install, completely independent of the release. Created with the `--thick|-T` option.
- Clone jails are essentially clones of the bootstrapped release. Changes to the release will affect the clone jail. Created with the `--clone|-C` option.

- Empty jails are just that, empty. These should be used only if you know what you are doing. Created with the `--empty|-E` option.
- Linux jails are jails that run linux. Created with the `--linux|-L` option. See *Linux Jails*.

We will focus on thin jails for this guide.

3.3.1 Classic/Standard Jail

```
ishmael ~ # bastille create nextcloud 14.2-RELEASE 10.1.1.4/24
```

This will create a classic jail, which uses the loopback interface (created with `bastille setup`) for outbound connections.

To be able to reach a service inside the jail, use `bastille rdr`.

```
ishmael ~ # bastille rdr nextcloud tcp 80 80
```

This will forward traffic from port 80 on the host to port 80 inside the jail. See also *rdr*.

3.3.2 VNET Jail

VNET jails can use either a host interface with `-V` or a manually created bridge interface with `-B`. You can also optionally set a static MAC for the jail interface with `-M`.

```
ishmael ~ # bastille create -BM nextcloud 14.2-RELEASE 192.168.1.50/24 bridge0
```

or

```
ishmael ~ # bastille create -VM nextcloud 14.2-RELEASE 192.168.1.50/24 vtnet0
```

The IP used for VNET jails should be an IP reachable inside your local network. You can also specify 0.0.0.0 or DHCP to use DHCP.

CHAPTER

FOUR

CONFIGURATION

Bastille is configured using a default config file located at `/usr/local/etc/bastille/bastille.conf`. When first installing bastille, you should run `bastille setup`. This will ask if you want to copy the sample config file to the above location. The defaults are sensible for UFS, but if you use ZFS, `bastille setup` will configure it for you. If you have multiple zpools, Bastille will ask which one you want to use. See also *ZFS Support*.

This is the default *bastille.conf* file.

```
####################
## [ BastilleBSD ] ##
####################

## Default paths
bastille_prefix="/usr/local/bastille"                                    ## default: "/usr/
↪local/bastille"
bastille_backupsdir="${bastille_prefix}/backups"                         ## default: "$
↪{bastille_prefix}/backups"
bastille_cachedir="${bastille_prefix}/cache"                             ## default: "$
↪{bastille_prefix}/cache"
bastille_jailsdir="${bastille_prefix}/jails"                             ## default: "$
↪{bastille_prefix}/jails"
bastille_releasesdir="${bastille_prefix}/releases"                       ## default: "$
↪{bastille_prefix}/releases"
bastille_templatesdir="${bastille_prefix}/templates"                     ## default: "$
↪{bastille_prefix}/templates"
bastille_logsdir="/var/log/bastille"                                     ## default: "/var/
↪log/bastille"

## PF firewall configuration path
bastille_pf_conf="/etc/pf.conf"                                          ## default: "/etc/
↪pf.conf"

## Bastille commands directory (assumed by bastille pkg)
bastille_sharedir="/usr/local/share/bastille"                            ## default: "/usr/
↪local/share/bastille"

## Bootstrap archives, which components of the OS to install.
## base  - The base OS, kernel + userland
## lib32 - Libraries for compatibility with 32 bit binaries
## ports - The FreeBSD ports (3rd party applications) tree
## src   - The source code to the kernel + userland
## test  - The FreeBSD test suite
```

(continues on next page)

(continued from previous page)

```
## Whitespace-separated list:
## bastille_bootstrap_archives="base lib32 ports src test"
bastille_bootstrap_archives="base"                                    ## default: "base"

## Pkgbase package sets
## Any set with [-dbg] can be installed with debugging
## symbols by adding '-dbg' to the package set
## base[-dbg]          - Base system
## base-jail[-dbg]     - Base system for jails
## devel[-dbg]         - Development tools
## kernels[-dbg]       - Base system kernels
## lib32[-dbg]         - 32-bit compatability libraries
## minimal[-dbg]       - Basic multi-user system
## minimal-jail[-dbg]  - Basic multi-user jail system
## optional[-dbg]      - Optional base system software
## optional-jail[-dbg] - Optional base system software for jails
## src                 - System source code
## tests               - System test suite
## Whitespace-separated list:
## bastille_pkgbase_packages="base-jail lib32-dbg src"
bastille_pkgbase_packages="base-jail"                                 ## default: "base-
↪jail"

## Default timezone
bastille_tzdata=""                                                    ## default: empty␣
↪to use host's time zone

## Default jail resolv.conf
bastille_resolv_conf="/etc/resolv.conf"                               ## default: "/etc/
↪resolv.conf"

## Bootstrap URLs
bastille_url_freebsd="http://ftp.freebsd.org/pub/FreeBSD/releases/"                  ##␣
↪default: "http://ftp.freebsd.org/pub/FreeBSD/releases/"
bastille_url_hardenedbsd="https://installers.hardenedbsd.org/pub/"                  ##␣
↪default: "https://installer.hardenedbsd.org/pub/HardenedBSD/releases/"
bastille_url_midnightbsd="https://www.midnightbsd.org/ftp/MidnightBSD/releases/"    ##␣
↪default: "https://www.midnightbsd.org/pub/MidnightBSD/releases/"

## ZFS options
bastille_zfs_enable="NO"                                              ## default: "NO"
bastille_zfs_zpool=""                                                 ## default: ""
bastille_zfs_prefix="bastille"                                        ## default:
↪"bastille"
bastille_zfs_options="-o compress=on -o atime=off"                    ## default: "-o␣
↪compress=on -o atime=off"

## Export/Import options
bastille_compress_xz_options="-0 -v"                                  ## default "-0 -v"
bastille_decompress_xz_options="-c -d -v"                             ## default "-c -d -
↪v"
bastille_compress_gz_options="-1 -v"                                  ## default "-1 -v"
```

(continues on next page)

(continued from previous page)

```
bastille_decompress_gz_options="-k -d -c -v"                                    ## default "-k -d -
↪c -v"
bastille_compress_zst_options="-3 -v"                                           ## default "-3 -v"
bastille_decompress_zst_options="-k -d -c -v"                                   ## default "-k -d -
↪c -v"
bastille_export_options=""                                                      ## default ""␣
↪predefined export options, e.g. "--live --gz"

## Networking
bastille_network_vnet_type="if_bridge"                                          ## default: "if_
↪bridge"
bastille_network_loopback="bastille0"                                           ## default:
↪"bastille0"
bastille_network_pf_ext_if="ext_if"                                             ## default: "ext_if
↪"
bastille_network_pf_table="jails"                                               ## default: "jails"
bastille_network_shared=""                                                      ## default: ""
bastille_network_gateway=""                                                     ## default: ""
bastille_network_gateway6=""                                                    ## default: ""

## Default Templates
bastille_template_base="default/base"                                           ## default:
↪"default/base"
bastille_template_empty=""                                                      ## default:
↪"default/empty"
bastille_template_thick="default/thick"                                         ## default:
↪"default/thick"
bastille_template_clone="default/clone"                                         ## default:
↪"default/clone"
bastille_template_thin="default/thin"                                           ## default:
↪"default/thin"
bastille_template_vnet="default/vnet"                                           ## default:
↪"default/vnet"
bastille_template_vlan="default/vlan"                                           ## default:
↪"default/vlan"

## Monitoring
bastille_monitor_cron_path="/usr/local/etc/cron.d/bastille-monitor"                                ␣
↪     ## default: "/usr/local/etc/cron.d/bastille-monitor"
bastille_monitor_cron="*/5 * * * * root /usr/local/bin/bastille monitor ALL >/dev/null 2>
↪&1"  ## default: "*/5 * * * * root /usr/local/bin/bastille monitor ALL >/dev/null 2>&1"
bastille_monitor_logfile="${bastille_logsdir}/monitor.log"                                         ␣
↪     ## default: "${bastille_logsdir}/monitor.log"
bastille_monitor_healthchecks=""                                                                   ␣
↪     ## default: ""
```

4.1 Notes

The options here are fairly self-explanitory, but there are some things to note.

- Bastille will mount the dataset it creates at `bastille_prefix` which defaults to `/usr/local/bastille`. So if you want to navigate to your jails, you will use the `bastille_prefix` as the location because this is where

the will be mounted.

4.2 Custom Configuration

Bastille supports using a custom config in addition to the default one. This is nice if you have multiple users, or want to store different jails at different locations based on your needs.

The customized config file MUST BE PLACED INSIDE THE BASTILLE CONFIG FOLDER at `/usr/local/etc/bastille` or it will not work.

Simply copy the default config file and edit it according to your new environment or user. Then, it can be used in a couple of ways.

1. Run Bastille using `bastille --config config.conf bootstrap 14.2-RELEASE` to bootstrap the release using the new config.
2. As a specific user, export the `BASTILLE_CONFIG` variable using `export BASTILLE_CONFIG=config.conf`. This config will then always be used when running Bastille with that user. See notes below...

- Exporting the `BASTILLE_CONFIG` variable will only export it for the current session. If you want to persist the export, see documentation for the shell that you use.
- If you use sudo, you will need to run it with `sudo -E bastille bootstrap...` to preserve your users environment. This can also be persisted by editing the sudoers file.
- If you do set the `BASTILLE_CONFIG` variable, you do not need to specify the config file when running Bastille as that specified user.

Note: FreeBSD introduced container technology twenty years ago, long before the industry standardized on the term "container". Internally, FreeBSD refers to these containers as "jails".

4.3 Jail Startup Configuration

Bastille can start jails on system startup, and stop them on system shutdown. To enable this functionality, we must first enable Bastille as a service using `sysrc bastille_enable=YES`. Once you reboot your host, all jails with `boot=on` will be started when the host boots.

If you have certain jails that must be started before other jails, you can use the priority option. Jails will start in order starting at the lowest value, and will stop in order starting at the highest value. So, jails with a priority value of 1 will start first, and stop last.

See *Targeting* for more info.

4.3.1 Boot

The boot setting controls whether a jail will be started on system startup. If you have enabled bastille with `sysrc bastille_enable=YES`, all jails with `boot=on` will start on system startup. Any jail(s) with `boot=off` will not be started on system startup.

By default, when jails are created with Bastille, the boot setting is set to `on` by default. This can be overridden using the `--no-boot` flag. See `bastille create --no-boot TARGET...`.

You can also use `bastille start --boot TARGET` to make Bastille respect the boot setting. If `-b|--boot` is not used, the targeted jail(s) will start, regardless of the boot setting.

Jails will still shut down on system shutdown, regardless of this setting.

The `-b|--boot` can also be used with the `stop` command. Any jails with `boot=off` will not be touched if `stop` is called with `-b|--boot`. Same goes for the `restart` command.

This value can be changed using `bastille config TARGET set boot [on|off]`.

This value will be shown using `bastille list all`.

4.3.2 Depend

Bastille supports configuring jails to depend on each other when started and stopped. If jail1 "depends" on jail2, then jail2 will be started if it is not running when `bastille start jail1` is called. Any jail that jail1 "depends" on will first be verified running (started if stopped) before jail1 is started.

For example, I have 3 jails called nginx, mariadb and nextcloud. I want to ensure that nginx and mariadb are running before nextcloud is started.

First we must add both jails to nextcloud's depend property with `bastille config nextcloud set depend "mariadb nginx"`. Then, when we start nextcloud with `bastille start nextcloud` it will verify that nginx and mariadb are running (start if stopped) before starting nextcloud.

When stopping a jail, any jail that "depends" on it will first be stopped. For example, if we run `bastille stop nginx`, then nextcloud will first be stopped because it "depends" on nginx.

Note that if we do a `bastille restart nginx`, however, nextcloud will be stopped, because it "depends" on nginx, but will not be started again, because the jail we just restarted, nginx, does not depend on nextcloud.

4.3.3 Parallel Startup

Bastille supports starting, stopping and restarting jails in parallel mode using the `rc` service script. To enable this functionality, set `bastille_parallel_limit` to a numeric value.

For example, if you run `sysrc bastille_parallel_limit=4`, then Bastille will start 4 jails at a time on system startup, as well as stop or restart 4 jails at a time when `service bastille...` is called.

This value is set to 1 by default, to only start/stop/restart jails one at a time.

4.3.4 Startup Delay

Sometimes it is necessary to let a jail start fully before continuing to the next jail.

We can do this with another sysrc value called `bastille_startup_delay`. Setting `bastille_startup_delay=5` will tell Bastille to wait 5 seconds between starting each jail.

You can also use `bastille start -d|--delay 5 all` or `bastille restart -d|--delay 5 all` to achieve the same thing.

4.4 jail.conf

In this section we'll look at the default config for a new container. The defaults are sane for most applications, but if you want to tweak the settings here they are.

A `jail.conf` template is used each time a new container is created. This template looks like this:

```
{name} {
  devfs_ruleset = 4;
  enforce_statfs = 2;
  exec.clean;
  exec.consolelog = /var/log/bastille/{name}_console.log;
  exec.start = '/bin/sh /etc/rc';
  exec.stop = '/bin/sh /etc/rc.shutdown';
  host.hostname = {name};
```

(continues on next page)

(continued from previous page)

```
  interface = {interface};
  mount.devfs;
  mount.fstab = /usr/local/bastille/jails/{name}/fstab;
  path = /usr/local/bastille/jails/{name}/root;
  securelevel = 2;

  ip4.addr = interface|x.x.x.x;
  ip6 = disable;
}
```

4.4.1 devfs_ruleset

```
devfs_ruleset
  The number of the devfs ruleset that is enforced for mounting
  devfs in this jail.  A value of zero (default) means no ruleset
  is enforced.  Descendant jails inherit the parent jail's devfs
  ruleset enforcement.  Mounting devfs inside a jail is possible
  only if the allow.mount and allow.mount.devfs permissions are
  effective and enforce_statfs is set to a value lower than 2.
  Devfs rules and rulesets cannot be viewed or modified from inside
  a jail.

  NOTE: It is important that only appropriate device nodes in devfs
  be exposed to a jail; access to disk devices in the jail may
  permit processes in the jail to bypass the jail sandboxing by
  modifying files outside of the jail.  See devfs(8) for
  information on how to use devfs rules to limit access to entries
  in the per-jail devfs.  A simple devfs ruleset for jails is
  available as ruleset #4 in /etc/defaults/devfs.rules.
```

4.4.2 enforce_statfs

```
enforce_statfs
  This determines what information processes in a jail are able to
  get about mount points.  It affects the behaviour of the
  following syscalls: statfs(2), fstatfs(2), getfsstat(2), and
  fhstatfs(2) (as well as similar compatibility syscalls).  When
  set to 0, all mount points are available without any
  restrictions.  When set to 1, only mount points below the jail's
  chroot directory are visible.  In addition to that, the path to
  the jail's chroot directory is removed from the front of their
  pathnames.  When set to 2 (default), above syscalls can operate
  only on a mount-point where the jail's chroot directory is
  located.
```

4.4.3 exec.clean

```
exec.clean
  Run commands in a clean environment.  The environment is
  discarded except for HOME, SHELL, TERM and USER.  HOME and SHELL
```

(continues on next page)

(continued from previous page)

```
  are set to the target login's default values.  USER is set to the
  target login.  TERM is imported from the current environment.
  The environment variables from the login class capability
  database for the target login are also set.
```

4.4.4 exec.consolelog

```
exec.consolelog
  A file to direct command output (stdout and stderr) to.
```

4.4.5 exec.start

```
exec.start
  Command(s) to run in the jail environment when a jail is created.
  A typical command to run is "sh /etc/rc".
```

4.4.6 exec.stop

```
exec.stop
  Command(s) to run in the jail environment before a jail is
  removed, and after any exec.prestop commands have completed.  A
  typical command to run is "sh /etc/rc.shutdown".
```

4.4.7 host.hostname

```
host.hostname
  The hostname of the jail.  Other similar parameters are
  host.domainname, host.hostuuid and host.hostid.
```

4.4.8 mount.devfs

```
mount.devfs
  Mount a devfs(5) filesystem on the chrooted /dev directory, and
  apply the ruleset in the devfs_ruleset parameter (or a default of
  ruleset 4: devfsrules_jail) to restrict the devices visible
  inside the jail.
```

4.4.9 mount.fstab

```
mount.fstab
  An fstab(5) format file containing filesystems to mount before
  creating a jail.
```

4.4.10 path

```
path
  The directory which is to be the root of the jail.  Any commands
```

(continues on next page)

(continued from previous page)

```
  run inside the jail, either by jail or from jexec(8), are run
  from this directory.
```

4.4.11 securelevel

By default, Bastille containers run at `securelevel = 2;`. See below for the implications of kernel security levels and when they might be altered.

Note: Bastille does not currently have any mechanism to automagically change securelevel settings. My recommendation is this only be altered manually on a case-by-case basis and that "Highly secure mode" is a sane default for most use cases.

```
The kernel runs with five different security levels.  Any super-user
process can raise the level, but no process can lower it.  The security
levels are:

-1    Permanently insecure mode - always run the system in insecure mode.
      This is the default initial value.

0     Insecure mode - immutable and append-only flags may be turned off.
      All devices may be read or written subject to their permissions.

1     Secure mode - the system immutable and system append-only flags may
      not be turned off; disks for mounted file systems, /dev/mem and
      /dev/kmem may not be opened for writing; /dev/io (if your platform
      has it) may not be opened at all; kernel modules (see kld(4)) may
      not be loaded or unloaded.  The kernel debugger may not be entered
      using the debug.kdb.enter sysctl.  A panic or trap cannot be forced
      using the debug.kdb.panic and other sysctl's.

2     Highly secure mode - same as secure mode, plus disks may not be
      opened for writing (except by mount(2)) whether mounted or not.
      This level precludes tampering with file systems by unmounting
      them, but also inhibits running newfs(8) while the system is multi-
      user.

      In addition, kernel time changes are restricted to less than or
      equal to one second.  Attempts to change the time by more than this
      will log the message "Time adjustment clamped to +1 second".

3     Network secure mode - same as highly secure mode, plus IP packet
      filter rules (see ipfw(8), ipfirewall(4) and pfctl(8)) cannot be
      changed and dummynet(4) or pf(4) configuration cannot be adjusted.
```

CHAPTER

FIVE

TARGETING

Bastille uses a `subcommand TARGET ARGS` syntax, meaning that each command requires a target. Targets are usually jails, but can also be releases.

Targeting a jail is done by providing the exact jail name, the JID of the jail, a tag, or by typing the starting few characters of a jail.

If you use a tag as the TARGET, Bastille will target any and all jails that have that tag assigned. If you have a jail with the same name as the tag you are trying to target, Bastille will target the jail, and not the tag.

Targeting a release is done by providing the exact release name. (Note: do not include the `-pX` point-release version.)

Bastille includes a pre-defined keyword of [ALL|all] to target all running jails. It is also possible to target multiple jails by grouping them in quotes, as seen below.

```
ishmael ~ # bastille cmd "jail1 jail2 jail3" echo Hello!
```

5.1 Priority

The priority value determines in what order commands are executed if multiple jails are targetted, including the [ALL|all] target.

It also controls in what order jails are started and stopped on system startup and shutdown. This requires Bastille to be enabled with `sysrc bastille_enable=YES`. Jails will start in order starting at the lowest value, and will stop in order starting at the highest value. So, jails with a priority value of 1 will start first, and stop last.

When jails are created with Bastille, this value defaults to `99`, but can be overridden with `-p|--priority VALUE` on creation. See `bastille create --priority 90 TARGET...`.

This value can be changed using `bastille config TARGET set priority VALUE`.

This value will be shown using `bastille list all`.

5.2 Examples: Jails

```
ishmael ~ # bastille ...
```

command	target	args	description
cmd	ALL	'sockstat -4'	execute *sockstat -4* in ALL jails (ip4 sockets)
console	mariadb02	—	console (shell) access to mariadb02
pkg	web01	'install nginx'	install nginx package in web01 jail
pkg	ALL	upgrade	upgrade packages in ALL jails
pkg	ALL	audit	(CVE) audit packages in ALL jails
sysrc	web01	nginx_enable=YES	execute *sysrc nginx_enable=YES* in web01 jail
template	ALL	username/base	apply *username/base* template to ALL jails
start	web02	—	start web02 jail
cp	bastion03	/tmp/resolv.conf-cf etc/resolv.conf	copy host-path to jail-path in bastion03
create	folsom	13.2-RELEASE 10.17.89.10	create 13.2 jail named *folsom* with IP

5.3 Examples: Releases

```
ishmael ~ # bastille ...
```

command	target	args	description
bootstrap	13.2-RELEASE	—	bootstrap 13.2-RELEASE release
update	12.4-RELEASE	—	update 12.4-RELEASE release
verify	12.4-RELEASE	—	verify 12.4-RELEASE release

CHAPTER

SIX

BASTILLE SUB-COMMANDS

6.1 bootstrap

The bootstrap sub-command is used to download and extract releases and templates for use with Bastille jail. A valid release is needed before jails can be created. Templates are optional but are managed in the same manner.

Note: your mileage may vary with unsupported releases, and releases newer than the host system likely will NOT work at all. Bastille tries to filter for valid release names, but if you find it will not bootstrap a valid release, please let us know.

6.1.1 Releases

Example

To `bootstrap` a FreeBSD release, run the bootstrap sub-command with the release version as the argument.

```
ishmael ~ # bastille bootstrap 15.0-RELEASE
ishmael ~ # bastille bootstrap 14.4-RELEASE
```

To `bootstrap` a HardenedBSD release, run the bootstrap sub-command with the build version as the argument.

```
ishmael ~ # bastille bootstrap 15-stable
```

This command will ensure the required directory structures are in place and download the requested release. For each requested release, `bootstrap` will download the base.txz. These files are verified (sha256 via MANIFEST file) before they are extracted for use.

EOL Releases

It is sometimes necessary to run end-of-life releases for testing or legacy application support. By default Bastille will only install supported releases but you can bootstrap EOL / unsupported releases with a simple trick.

```
ishmael ~ # export BASTILLE_URL_FREEBSD=http://ftp-archive.freebsd.org/pub/FreeBSD-
↪Archive/old-releases/
ishmael ~ # bastille bootstrap 11.2-RELEASE
```

By overriding the BASTILLE_URL_FREEBSD variable you can now bootstrap archived releases from the FTP archive.

Tips

The `bootstrap` sub-command can now take (0.5.20191125+) an optional second argument of `update`. If this argument is used, `bastille update` will be run immediately after the bootstrap, effectively bootstrapping and applying security patches and errata in one motion.

Notes

The bootstrap subcommand is generally only used once to prepare the system. The only other use case for the bootstrap command is when a new FreeBSD version is released and you want to start deploying containers on that version.

To update a release as patches are made available, see the `bastille update` command.

Downloaded artifacts are stored in the `bastille/cache/version` directory. "bootstrapped" releases are stored in `bastille/releases/version`.

To manually bootstrap a release (aka bring your own archive), place your archive in bastille/cache/name and extract to bastille/releases/name. Your mileage may vary; let me know what happens.

6.1.2 Templates

Bastille aims to integrate container automation into the platform while maintaining a simple, uncomplicated design. Templates are git repositories with automation definitions for packages, services, file overlays, etc.

To download one of these templates see the example below.

Example

```
ishmael ~ # bastille bootstrap https://gitlab.com/bastillebsd-templates/nginx
ishmael ~ # bastille bootstrap https://gitlab.com/bastillebsd-templates/mariadb-server
ishmael ~ # bastille bootstrap https://gitlab.com/bastillebsd-templates/python3
```

Tips

See the documentation on templates for more information on how they work and how you can create or customize your own. Templates are a powerful part of Bastille and facilitate full container automation.

Notes

If you don't want to bother with git to use templates you can create them manually on the Bastille system and apply them.

Templates are stored in `bastille/templates/namespace/name`. If you'd like to create a new template on your local system, simply create a new namespace within the templates directory and then one for the template. This namespacing allows users and groups to have templates without conflicting template names.

Once you've created the directory structure you can begin filling it with template hooks. Once you have a minimum number of hooks (at least one) you can begin applying your template.

```
ishmael ~ # bastille bootstrap help
Usage: bastille bootstrap [option(s)] RELEASE [update|ARCH]
                                      TEMPLATE

    Options:

    -p | --pkgbase     Bootstrap using pkgbase (FreeBSD 15.0-RELEASE and above).
    -u | --update      Update the release after bootstrap.
    -x | --debug       Enable debug mode.
```

6.2 clone

6.2.1 Limitations

- When cloning a vnet jail with multiple interfaces, the default interface will be assigned the IP given in the command. The rest of the interfaces will have their network info set to `ifconfig_inet=""`. This is to avoid conflicts between the old and new jails.

```
ishmael ~ # bastille clone help
Usage: bastille clone [option(s)] TARGET NEW_NAME IP

    Options:

    -a | --auto      Auto mode. Start/stop jail(s) if required. Cannot be used with [-l|-
↪-live].
    -l | --live      Clone a running jail (ZFS only). Cannot be used with [-a|--auto].
    -x | --debug     Enable debug mode.
```

6.3 cmd

```
ishmael ~ # bastille cmd folsom ps -auxw
[folsom]:
USER   PID %CPU %MEM   VSZ  RSS TT  STAT STARTED    TIME COMMAND
root 71464  0.0  0.0 14536 2000  -  IsJ   4:52PM 0:00.00 /usr/sbin/syslogd -ss
root 77447  0.0  0.0 16632 2140  -  SsJ   4:52PM 0:00.00 /usr/sbin/cron -J 60 -s
root 80591  0.0  0.0 18784 2340  1  R+J   4:53PM 0:00.00 ps -auxw
```

```
ishmael ~ # bastille cmd help
Usage: bastille cmd [option(s)] TARGET COMMAND

    Options:

    -a | --auto      Auto mode. Start/stop jail(s) if required.
    -x | --debug     Enable debug mode.
```

6.4 config

Getting a property that *is* defined in jail.conf:

```
ishmael ~ # bastille config azkaban get ip4.addr
bastille0|192.168.2.23
```

Getting a property that *is not* defined in jail.conf

```
ishmael ~ # bastille config azkaban get notaproperty
not set
```

Setting a property:

```
ishmael ~ # bastille config azkaban set allow.mlock 1
A restart is required for the changes to be applied. See 'bastille restart azkaban'.
```

The restart message will appear every time a property is set.

Removing a property:

```
ishmael ~ # bastille config azkaban remove allow.mlock
A restart is required for the changes to be applied. See 'bastille restart azkaban'.
```

The restart message will appear every time a property is removed.

```
ishmael ~ # bastille config help
Usage: bastille config [option(s)] TARGET set|add PROPERTY [VALUE]
                                          get|remove PROPERTY

    Options:

    -x | --debug     Enable debug mode.
```

6.5 console

Launch a login shell into the jail. Default is password- less root login.

```
ishmael ~ # bastille console folsom
[folsom]:
root@folsom:~ #
```

At this point you are logged in to the jail and have full shell access. The system is yours to use and/or abuse as you like. Any changes made inside the jail are limited to the jail.

```
ishmael ~ # bastille console help
Usage: bastille console [option(s)] TARGET [USER]

    Options:

    -a | --auto      Auto mode. Start/stop jail(s) if required.
    -x | --debug     Enable debug mode.
```

6.6 convert

Converting a thin jail to a thick jail requires only the TARGET arg.

```
ishmael ~ # bastille convert azkaban
```

Converting a thick jail to a custom release requires the TARGET and RELEASE as args.

```
ishmael ~ # bastille convert azkaban myrelease
```

This release can then be used to create a thick jail using the `--no-validate` flag.

```
ishmael ~ # bastille create --no-validate customjail myrelease 10.0.0.1
```

```
ishmael ~ # bastille convert help
Usage: bastille convert [option(s)] TARGET
```

(continues on next page)

(continued from previous page)

```
                                    TARGET RELEASE

    Options:

    -a | --auto      Auto mode. Start/stop jail(s) if required.
    -y | --yes       Do not prompt. Assume always yes.
    -x | --debug     Enable debug mode.
```

6.7 cp

```
ishmael ~ # bastille cp ALL /tmp/resolv.conf-cf /etc/resolv.conf
[bastion]:
/tmp/resolv.conf-cf -> /usr/local/bastille/jails/bastion/root/etc/resolv.conf
[unbound0]:
/tmp/resolv.conf-cf -> /usr/local/bastille/jails/unbound0/root/etc/resolv.conf
[unbound1]:
/tmp/resolv.conf-cf -> /usr/local/bastille/jails/unbound1/root/etc/resolv.conf
[squid]:
/tmp/resolv.conf-cf -> /usr/local/bastille/jails/squid/root/etc/resolv.conf
[nginx]:
/tmp/resolv.conf-cf -> /usr/local/bastille/jails/nginx/root/etc/resolv.conf
[folsom]:
/tmp/resolv.conf-cf -> /usr/local/bastille/jails/folsom/root/etc/resolv.conf
```

Unless you see errors reported in the output the `cp` was successful.

```
ishmael ~ # bastille cp help
Usage: bastille cp [option(s)] TARGET HOST_PATH JAIL_PATH

    Options:

    -q | --quiet     Suppress output.
    -x | --debug     Enable debug mode.
```

6.8 create

Create a jail uning any available bootstrapped release. To create a jail, simply provide a name, bootstrapped release, and IP address.

The format is `bastille create NAME RELEASE IP [INTERFACE]`

Note that the `interface` is optional. Bastille will use the default interface that is configured when running the setup command. See `bastille setup -l` or `bastille setup -s`.

```
ishmael ~ # bastille create folsom 11.3-RELEASE 10.17.89.10 [INTERFACE]

RELEASE: 11.3-RELEASE.
NAME: folsom.
IP: 10.17.89.10.
```

This command will create a 11.3-RELEASE jail, assigning the 10.17.89.10 ip address to the new jail.

```
ishmael ~ # bastille create alcatraz 13.2-RELEASE 10.17.89.113/24
```

The above code will create a jail with a /24 mask. At the time of this documentation you can only use CIDR notation, and not use a netmask 255.255.255.0 to accomplish this.

I recommend using private (rfc1918) ip address ranges for your container. These ranges include:

- 10.0.0.0/8 - 172.16.0.0/12 - 192.168.0.0/16

Bastille does its best to validate the submitted ip is valid. This has not been thouroughly tested. I generally use the 10/8 range.

A couple of notes about the created jails. First, MOTD has been disabled inside of the jails because it does not give information about the jail, but about the host system. This caused confusion for some users, so we implemented the .hushlogin which silences the MOTD at login.

Also, uname does not work from within a jail. Much like MOTD, it gives you the version information about the host system instead of the jail. If you need to check the version of freebsd running on the jail use the freebsd-version command to get accurate information.

Bastille can create many different types of jails, along with many different options. See the below help output.

```
ishmael ~ # bastille create help
Usage: bastille create [option(s)] NAME RELEASE IP [INTERFACE]"

    Options:

    -B | --bridge                   Enable VNET. INTERFACE must be a bridge.
    -C | --clone                    Create a clone jail (ZFS only).
    -D | --dual                     Use dual (IPv4+6) networking (IP=[inherit|ip_
↪hostname] only).
    -E | --empty                    Create an empty jail (NAME only).
    -g | --gateway IP               Specify a default router/gateway.
    -L | --linux                    Create a Linux jail (experimental).
    -M | --static-mac               Use a static/persistent MAC address (VNET only).
    -n | --nameserver IP            Specify nameserver(s) for the jail. Comma-separated.
         --no-boot                  Set boot=off.
         --no-validate              Do not validate the release name.
         --no-ip                    Create jail without an ip (VNET only).
    -P | --passthrough              Enable VNET. INTERFACE is used as-is.
    -p | --priority VALUE           Set priority value.
         --tags TAG1,TAG2           Apply specified tag(s) to jail. Comma-separated.
    -T | --thick                    Create a thick jail.
    -V | --vnet                     Enable VNET. INTERFACE must be a physical interface.
    -v | --vlan VLANID              Set VLAN ID (VNET only).
    -x | --debug                    Enable debug mode.
    -Z | --zfs-opts zfs,options     Custom zfs options. Comma-separated.
```

6.9 destroy

Bastille will normally ask if you are sure you want to delete targeted jail(s). Use the `-y|--yes` option to bypass this prompt.

```
ishmael ~ # bastille destroy -a folsom
[folsom]:
```

(continues on next page)

(continued from previous page)

```
folsom: removed
Deleting Jail: folsom.
Note: jail console logs archived.
/var/log/bastille/folsom_console.log-YYYY-MM-DD
```

Release can be destroyed provided there are no child jails. The `-c|--no-cache` option will retain the release cache (`*.txz` file), if you choose to keep it.

```
ishmael ~ # bastille destroy help
Usage: bastille destroy [option(s)] JAIL
                                    RELEASE

    Options:

    -a | --auto         Auto mode. Start/stop jail(s) if required.
    -c | --no-cache     Do not destroy cache when destroying a release (legacy releases).
    -f | --force        Force unmount any mounted datasets when destroying a jail or␣
↪release (ZFS only).
    -y | --yes          Do not prompt. Assume always yes.
    -x | --debug        Enable debug mode.
```

6.10 edit

```
ishmael ~ # bastille edit azkaban [FILE]
```

Syntax requires a target an optional filename. By default the file edited will be `jail.conf`. Other common filenames are `fstab` or `rctl.conf`.

```
ishmael ~ # bastille edit help
Usage: bastille edit [option(s)] TARGET [FILE]

    Options:

    -x | --debug          Enable debug mode.
```

6.11 etcupdate

This command will update the contents of `/etc` inside a jail. It should be run after a jail upgrade

First we need to bootstrap a release for `etcupdate` to use.

```
ishmael ~ # bastille etcupdate bootstrap 14.1-RELEASE
bastille_bootstrap_archives: base -> src
/usr/local/bastille/cache/14.1-RELEASE/MANIFES        1046  B 1134 kBps    00s
/usr/local/bastille/cache/14.1-RELEASE/src.txz         205 MB 2711 kBps 01m18s
bastille_bootstrap_archives: src -> base
Building tarball, please wait...
Etcupdate bootstrap complete: 14.1-RELEASE
```

Next we can use the `update` command to apply the update to the jail.

```
ishmael ~ # bastille etcupdate ishmael update 14.1-RELEASE
```

The output will show you which files were added, updated, changed, deleted, or have conflicts. To automatically resolve the conflicts, run the `resolve` command.

```
ishmael ~ # bastille etcupdate ishmael resolve
```

To show only the differences between the releases, use the `diff` command.

```
ishmael ~ # bastille etcupdate ishmael diff 14.1-RELEASE
```

```
ishmael ~ # bastille etcupdate help
Usage: bastille etcupdate [option(s)] bootstrap RELEASE
                                      TARGET update RELEASE
                                      TARGET diff|resolve

    Options:

    -d | --dry-run     Show output, but do not apply.
    -f | --force       Force a re-bootstrap of a RELEASE.
    -x | --debug       Enable debug mode.
```

6.12 export

Exporting a container creates an archive or image that can be sent to a different machine to be imported later. These exported archives can be used as container backups.

```
ishmael ~ # bastille export azkaban
```

The export sub-command supports both UFS and ZFS storage. ZFS based containers will use ZFS snapshots. UFS based containers will use `txz` archives and they can be exported only when the jail is not running.

```
Usage:  bastille export [option(s)] TARGET PATH
```

Available options are:

```
ishmael ~ # bastille export help
Usage: bastille export [option(s)] TARGET [PATH]

    Options:

    -a | --auto        Auto mode. Start/stop jail(s) if required.
    -l | --live        Export a running jail (ZFS only).
         --gz          Export to a '.gz' compressed image (ZFS only).
         --xz          Export to a '.xz' compressed image (ZFS only).
         --zst         Export to a '.zst' compressed image (ZFS only).
         --raw         Export to an uncompressed RAW image (ZFS only).
         --tgz         Export to a '.tgz' compressed archive.
         --txz         Export to a '.txz' compressed archive.
         --tzst        Export to a '.tzst' compressed archive.
    -v | --verbose     Enable verbose mode (ZFS only).
    -x | --debug       Enable debug mode.
```

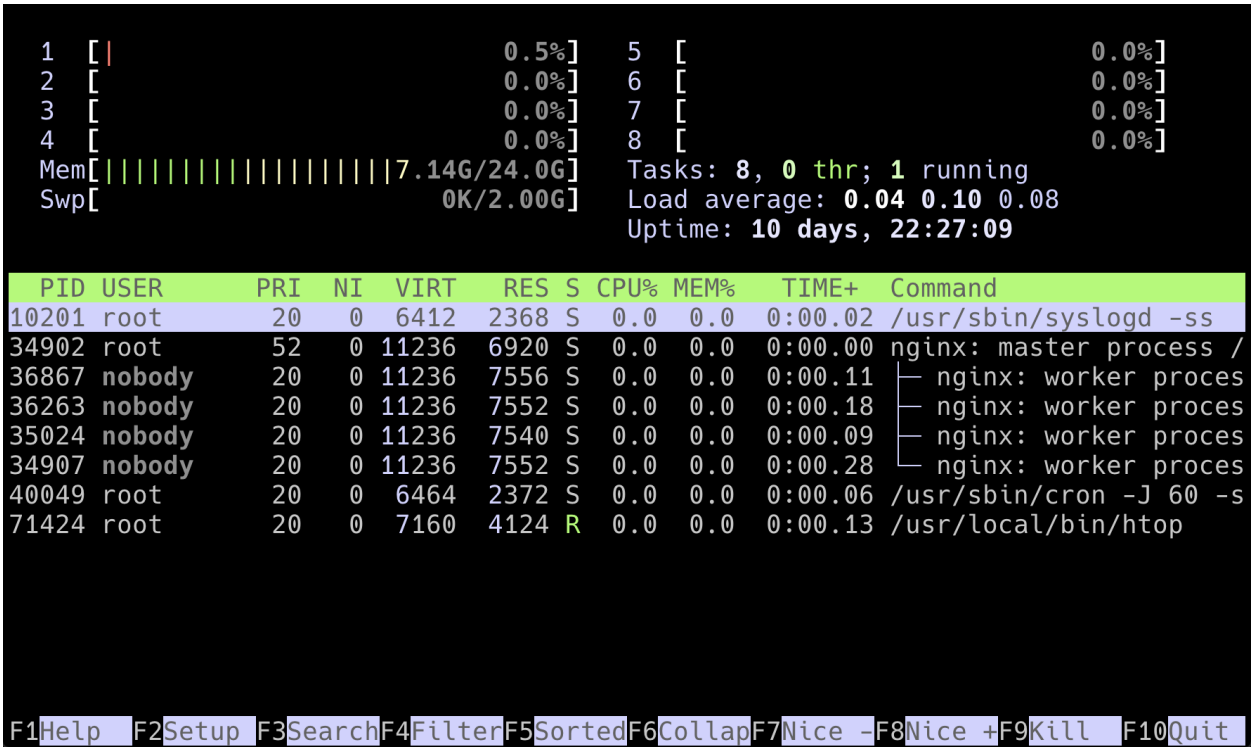
(continues on next page)

(continued from previous page)

```
   Note: If no export option specified, the container should be redirected to standard␣
↪output.
```

6.13 htop

This command runs `htop` in the targeted jail. Requires htop to be installed in the jail.




```
ishmael ~ # bastille htop help
Usage: bastille htop [options(s)] TARGET

    Options:

    -a | --auto      Auto mode. Start/stop jail(s) if required.
    -x | --debug     Enable debug mode.
```

6.14 import

```
ishmael ~ # bastille import /path/to/archive.file
```

The import sub-command supports both UFS and ZFS storage. ZFS based containers will use ZFS snapshots. UFS based containers will use `txz` archives.

To import to a specified release, specify it as the last argument.

```
ishmael ~ # bastille import help
Usage: bastille import [option(s)] FILE [RELEASE]

    Options:

    -f | --force          Force an archive import without validating checksum.
    -M | --static-mac     Use a static/persistent MAC address (VNET only) when importing␣
↪foreign jails.
    -v | --verbose        Enable verbose mode (ZFS only).
    -x | --debug          Enable debug mode.

    Tip: If no option specified, container should be imported from standard input.
```

6.15 jcp

```
ishmael ~ # bastille jcp bastion /tmp/resolv.conf-cf ALL /etc/resolv.conf
[unbound0]:
/usr/local/bastille/jails/bastion/root/tmp/resolv.conf-cf -> /usr/local/bastille/jails/
↪unbound0/root/etc/resolv.conf
[unbound1]:
/usr/local/bastille/jails/bastion/root/tmp/resolv.conf-cf -> /usr/local/bastille/jails/
↪unbound1/root/etc/resolv.conf
[squid]:
/usr/local/bastille/jails/bastion/root/tmp/resolv.conf-cf -> /usr/local/bastille/jails/
↪squid/root/etc/resolv.conf
[nginx]:
/usr/local/bastille/jails/bastion/root/tmp/resolv.conf-cf -> /usr/local/bastille/jails/
↪nginx/root/etc/resolv.conf
[folsom]:
/usr/local/bastille/jails/bastion/root/tmp/resolv.conf-cf -> /usr/local/bastille/jails/
↪folsom/root/etc/resolv.conf
```

Unless you see errors reported in the output the `jcp` was successful.

```
ishmael ~ # bastille jcp help
Usage: bastille jcp [option(s)] SOURCE_JAIL JAIL_PATH DESTINATION_JAIL JAIL_PATH

    Options:

    -q | --quiet     Suppress output.
    -x | --debug     Enable debug mode.
```

6.16 limits

6.16.1 rctl

To add a limit, use `bastille limits TARGET add OPTION VALUE`.

To clear the limits from the system, use `bastille limits TARGET clear`.

To clear the limits, and remove the rctl.conf, so that limits will not be loaded on a restart, use `bastille limits TARGET reset`. This removes the `rctl.conf` file, and removes any active limits from the system.

To remove a limit, use `bastille limits TARGET remove OPTION`.

This file can be edited manually using `bastille edit TARGET rctl.conf`.

Supported actions are `add`, `remove`, `clear`, `reset`, `list`, `show`, and `stats`.

6.16.2 cpuset

Bastille supports limiting CPUs using `cpuset`. To limit a jail to a specific CPU, use `bastille limits TARGET cpu 2,3,4` where the value (2,3,4) is a comma-separated list of CPUs on your system. Bastille will validate the CPUs, and error if they are not available to be used.

To adjust the CPU limits, run `bastille limits TARGET cpu 1,2,3` again with a new set of CPU values. This will overwrite the `cpuset.conf` file. This will restrict the targetted jail(s) to the specified CPUs.

CPU limits are cleared when the jail is stopped, and loaded again on jail start, providing the CPU values are present in `cpuset.conf` inside the jail directory.

Supported actions are `add`, `remove`, `reset`, `list` and `show`.

This file can be edited manually using `bastille edit TARGET cpuset.conf`.

```
ishmael ~ # bastille limits help
Usage: bastille limits [option(s)] TARGET add OPTION VALUE
                                   TARGET remove OPTION"
                                   TARGET clear|reset|stats"
                                   TARGET list|show [active]"

    Example: bastille limits TARGET add memoryuse 1G
    Example: bastille limits TARGET add cpu 0,1,2

    Options:

    -a | --auto      Auto mode. Start/stop jail(s) if required.
    -l | --log       Enable logging for the specified rule (RCTL only).
    -x | --debug     Enable debug mode.
```

6.17 list

List jails, ports, releases, templates, logs, limits, exports and imports and much more managed by bastille. See the `help` output below.

Using *bastille list* without args will print all jails with the info we feel is most important.

Most options can be printed in JSON format by including the `-j|--json` flag. Use `-p|--pretty` to print in columns instead of rows.

```
ishmael ~ # bastille list help
Usage: bastille list [option(s)] [all|backup|export|import|ip|jail|limit]"
↪[log|path|port|priority|release|snapshot|state|template|type]"

    Options:

    -d | --down       List stopped jails only.
    -j | --json       List jails or sub-arg(s) in json format.
    -p | --pretty     Print JSON in columns. Must be used with -j|--json.
```

(continues on next page)

(continued from previous page)

```
    -u | --up          List running jails only.
    -x | --debug       Enable debug mode.
```

6.18 migrate

The `migrate` sub-command allows migrating the targeted jail(s) to another remote system. See the chapter on Migration.

This sub-command supports multiple targets.

Syntax for the remote system is `user@host`. You can also specify a non-default port by supplying it as in `user@host:port`.

```
ishmael ~ # bastille migrate help
Usage: bastille migrate [option(s)] TARGET USER@HOST[:PORT]

    Options:

    -a | --auto             Auto mode. Start/stop jail(s) if required.
    -b | --backup           Keep archives on remote system.
    -d | --destroy          Destroy local jail after migration.
       | --doas             Use 'doas' instead of 'sudo'.
    -k | --keyfile FILE     Specify an alternative private keyfile name. Must be in '~/.
↪ssh'
    -l | --live             Migrate a running jail (ZFS only).
    -p | --password         Use password based authentication.
    -x | --debug            Enable debug mode.
```

6.19 monitor

NEW in Bastille version 1.1.20250814

The `monitor` sub-command adds, removes, lists and enables/disables monitoring for container services.

6.19.1 Managing Bastille Monitor

To enable Bastille monitoring, run `bastille monitor enable`.

To disable Bastille monitoring, run `bastille monitor disable`.

We can always check if Bastille monitoring is active with `bastille monitor status`.

6.19.2 Managing Services

Bastille Monitor will attempt to monitor any services defined for any given container. If the service is stopped, Bastille will attempt to restart it. Everything is logged in `${bastille_monitor_logfile}`.

To have Bastille monitor a service, run `bastille monitor TARGET add SERVICE`. The `SERVICE` arg can also be a comma-separated list of services such as `bastille monitor TARGET add SERVICE1,SERVICE2`.

To remove a service from monitoring, we can run `bastille monitor TARGET delete SERVICE`. These can also be a comma-separated list.

To show all services that Bastille is monitoring, run `bastille monitor TARGET list`.

To list all jails that have a selected service defined for monitoring, run `bastille monitor TARGET list SERVICE`. This option only accepts a single `SERVICE`, and cannot be a comma-separated list.

If you run `bastille monitor TARGET`, without any args or actions, Bastille will run through the process of checking the status of each defined service, and attempt to start any that are stopped.

Services can also be manually added or removed by editing the `monitor` file inside the jail directory, but is not recommended unless you are an advanced user.

6.19.3 Configuration

The monitor sub-command is configurable via the `bastille.conf` file. See below for configuration defaults:

```
bastille_monitor_cron_path="/usr/local/etc/cron.d/bastille-monitor"
bastille_monitor_cron="*/5 * * * * root /usr/local/bin/bastille monitor ALL >/dev/null 2&
↪>1"
bastille_monitor_logfile="${bastille_logsdir}/monitor.log"
bastille_monitor_module=""
bastille_monitor_url=""
```

6.19.4 Alerting modules

Bastille currently supports two healthcheck modules. The `bastille_monitor_module` should be either `uptimekuma` or `healthchecks.io`.

The first alerting module to be supported is Health Checks (https://healthchecks.io), which is both a free SaaS service (up to 20 checks) and provides a self-hosted option (see `sysutils/py-healthchecks`). Just add the healthcheck url to the `bastille_monitor_url` variable.

The second is Uptime-Kuma, a self-hosted monitoring application. Add a new monitor of type "Push", then copy and paste the given url into the `bastille_monitor_url` variable. Do not include the optional parameters following the ? character. Only the url including the api key should be included.

6.19.5 Help

```
ishmael ~ # bastille monitor help
Usage: bastille monitor [option(s)] enable|disable|status
                                    TARGET add|delete|list service1,service2
                                    TARGET list [service]
                                    TARGET

  Options:

  -x | --debug      Enable debug mode.
```

6.20 mount

Syntax follows standard `/etc/fstab` format:

```
Usage: bastille mount TARGET HOST_PATH JAIL_PATH [filesystem_type options dump pass_
↪number]
```

The `options` string can include a comma-separated list of mount options, but must include one of (rw,ro,rq,sw,xx) according to fstab documentation.

Example: Mount a tmpfs filesystem with options.

```
ishmael ~ # bastille mount azkaban tmpfs tmp tmpfs rw,nosuid,mode=01777 0 0
Detected advanced mount type tmpfs
[azkaban]:
Added: tmpfs /usr/local/bastille/jails/azkaban/root/tmp tmpfs rw,nosuid,mode=01777 0 0
```

Example: Mount a nullfs filesystem

```
ishmael ~ # bastille mount azkaban /storage/foo media/foo nullfs ro 0 0
[azkaban]:
Added: /media/foo /usr/local/bastille/jails/azkaban/root/media/foo nullfs ro 0 0
ishmael ~ # bastille mount azkaban /storage/bar /media/bar nullfs ro 0 0
[azkaban]:
Added: /media/bar /usr/local/bastille/jails/azkaban/root/media/bar nullfs ro 0 0
```

Notice the JAIL_PATH format can be /media/foo or simply media/bar. The leading slash / is optional. The HOST_PATH howerver, must be the full path including the leading slash /.

It is also possible to mount individual files into a jail as seen below. Bastille will not mount if a file is already present at the specified mount point. If the jail file name does not match the host file name, bastille will treat the jail path as a directory, and mount the file underneath as seen in the second example below.

```
ishmael ~ # bastille mount azkaban /etc/rc.conf /mnt/etc/rc.conf nullfs ro 0 0
[azkaban]:
Added: /etc/rc.conf /usr/local/bastille/jails/azkaban/root/mnt/etc/rc.conf nullfs ro 0 0
ishmael ~ # bastille mount azkaban /etc/rc.conf /media/bar nullfs ro 0 0
[azkaban]:
Added: /etc/rc.conf usr/local/bastille/jails/azkaban/root/media/bar/rc.conf nullfs ro 0 0
```

It is also possible (but not recommended) to have spaces in the directories that are mounted. It is necessary to escape each space with a backslash and enclose the mount point in quotes “” as seen below. It is possible to do the same for the jail path, but again, not recommemded.

```
ishmael ~ # bastille mount azkaban "/storage/my\ directory\ with\ spaces" /media/foo␣
↪nullfs ro 0 0
[azkaban]:
Added: /storage/my\040directory\040with\040spaces /usr/local/bastille/jails/azkaban/root/
↪media/foo nullfs ro 0 0
```

```
ishmael ~ # bastille mount help
Usage: bastille mount [option(s)] TARGET HOST_PATH JAIL_PATH [FS_TYPE OPTIONS DUMP PASS_
↪NUMBER]
  Options:

  -a | --auto      Auto mode. Start/stop jail(s) if required.
  -x | --debug     Enable debug mode.
```

6.21 network

You can only add an interface once to a jail, with two exceptions.

1. For classic jails, you can add an interface as many times as you want, but each time with a different IP. All this does is add the IP as another alias on that interface. This is the default if no option is given. See help output below.

2. For VNET jails, if the `-v|--vlan` switch is given along with a numerical VLAN ID, Bastille will add the VLAN ID to the jail as a vnetX.X interface, depending on which interface is specified.

Bridges and VNET interfaces can be added to VNET jails, no matter if they were created with `-V` or `-B`.

If no option is given, Bastille will assume a standard/classic jail.

It is possible to passthrough an entire interface from the host to the jail using the `-P|--passthrough` option. This will make the interface fully available without the need for additional configuration. It will be available inside the jail just like it would be on the host. Adding an interface using this method will render it only available inside the jail. It will not be present on the host until the jail is stopped.

When cloning a jail that has a `-P|--passthrough` interface, you will have warnings when running both jails at the same time. The first jail to start will be assigned the interface, and since it will no longer be available to the host, it will not be possible to add it to the second jail. To solve this, you must manually remove the interface from the jail.conf file, or running `bastille network TARGET remove INTERFACE` while both jails are stopped.

```
ishmael ~ # bastille network help
Usage: bastille network [option(s)] TARGET add INTERFACE [IP]
                                    TARGET remove INTERFACE

    Options:

    -a | --auto            Start/stop jail(s) if required.
    -B | --bridge          Add a bridge interface.
    -M | --static-mac      Use a static/persistent MAC address (VNET only).
    -n | --no-ip           Create interface without an IP (VNET only).
    -P | --passthrough     Add a raw interface.
    -V | --vnet            Add a physical interface.
    -v | --vlan VLANID     Assign VLANID to interface (VNET only).
    -x | --debug           Enable debug mode.
```

6.22 pkg

```
ishmael ~ # bastille pkg folsom install vim-console git-lite zsh
[folsom]:
The package management tool is not yet installed on your system.
Do you want to fetch and install it now? [y/N]: y
...[snip]...

Number of packages to be installed: 10

The process will require 77 MiB more space.
17 MiB to be downloaded.

Proceed with this action? [y/N]: y
...[snip]...
```

The PKG sub-command can do more than just `install`. The expectation is that you can fully leverage the pkg manager. This means, `install`, `update`, `upgrade`, `audit`, `clean`, `autoremove`, etc…

```
ishmael ~ # bastille pkg ALL upgrade
[bastion]:
Updating pkg.bastillebsd.org repository catalogue...
```

(continues on next page)

(continued from previous page)

```
[bastion] Fetching meta.txz: 100%    560 B   0.6kB/s    00:01
[bastion] Fetching packagesite.txz: 100%  118 KiB 121.3kB/s    00:01
Processing entries: 100%
pkg.bastillebsd.org repository update completed. 493 packages processed.
All repositories are up to date.
Checking for upgrades (1 candidates): 100%
Processing candidates (1 candidates): 100%
Checking integrity... done (0 conflicting)
Your packages are up to date.

[unbound0]:
Updating pkg.bastillebsd.org repository catalogue...
[unbound0] Fetching meta.txz: 100%    560 B   0.6kB/s    00:01
[unbound0] Fetching packagesite.txz: 100%  118 KiB 121.3kB/s    00:01
Processing entries: 100%
pkg.bastillebsd.org repository update completed. 493 packages processed.
All repositories are up to date.
Checking for upgrades (0 candidates): 100%
Processing candidates (0 candidates): 100%
Checking integrity... done (0 conflicting)
Your packages are up to date.

[unbound1]:
Updating pkg.bastillebsd.org repository catalogue...
[unbound1] Fetching meta.txz: 100%    560 B   0.6kB/s    00:01
[unbound1] Fetching packagesite.txz: 100%  118 KiB 121.3kB/s    00:01
Processing entries: 100%
pkg.bastillebsd.org repository update completed. 493 packages processed.
All repositories are up to date.
Checking for upgrades (0 candidates): 100%
Processing candidates (0 candidates): 100%
Checking integrity... done (0 conflicting)
Your packages are up to date.

[squid]:
Updating pkg.bastillebsd.org repository catalogue...
[squid] Fetching meta.txz: 100%    560 B   0.6kB/s    00:01
[squid] Fetching packagesite.txz: 100%  118 KiB 121.3kB/s    00:01
Processing entries: 100%
pkg.bastillebsd.org repository update completed. 493 packages processed.
All repositories are up to date.
Checking for upgrades (0 candidates): 100%
Processing candidates (0 candidates): 100%
Checking integrity... done (0 conflicting)
Your packages are up to date.

[nginx]:
Updating pkg.bastillebsd.org repository catalogue...
[nginx] Fetching meta.txz: 100%    560 B   0.6kB/s    00:01
[nginx] Fetching packagesite.txz: 100%  118 KiB 121.3kB/s    00:01
Processing entries: 100%
pkg.bastillebsd.org repository update completed. 493 packages processed.
```

(continues on next page)

(continued from previous page)

```
All repositories are up to date.
Checking for upgrades (1 candidates): 100%
Processing candidates (1 candidates): 100%
The following 1 package(s) will be affected (of 0 checked):

Installed packages to be UPGRADED:
    nginx-lite: 1.23.0 -> 1.24.0_12,3

Number of packages to be upgraded: 1

315 KiB to be downloaded.

Proceed with this action? [y/N]: y
[nginx] [1/1] Fetching nginx-lite-1.14.1,2.txz: 100%  315 KiB 322.8kB/s    00:01
Checking integrity... done (0 conflicting)
[nginx] [1/1] Upgrading nginx-lite from 1.23.0 to 1.24.0_12,3...
===> Creating groups.
Using existing group 'www'.
===> Creating users
Using existing user 'www'.
[nginx] [1/1] Extracting nginx-lite-1.24.0_12: 100%
You may need to manually remove /usr/local/etc/nginx/nginx.conf if it is no longer␣
↪needed.
```

```
ishmael ~ # bastille pkg help
Usage: bastille pkg [option(s)] TARGET ARGS

    Options:

    -a | --auto      Auto mode. Start/stop jail(s) if required.
    -H | --host      Use host 'pkg' binary instead of jails.
    -y | --yes       Do not prompt. Assume always yes.
    -x | --debug     Enable debug mode.
```

6.23 rcp

```
ishmael ~ # bastille rcp bastion /test/testfile.txt /tmp/testfile.txt
[bastion]:
/usr/local/bastille/jails/bastion/root/test/testfile.txt -> /tmp/testfile.txt
```

Unless you see errors reported in the output the `rcp` was successful.

```
ishmael ~ # bastille rcp help
Usage: bastille rcp [option(s)] TARGET JAIL_PATH HOST_PATH

    Options:

    -q | --quiet     Suppress output.
    -x | --debug     Enable debug mode.
```

6.24 rdr

`bastille rdr` allows you to configure dynamic rdr rules for your containers without modifying pf.conf (assuming you are using the `bastille0` interface for a private network and have enabled `rdr-anchor 'rdr/*'` in /etc/pf.conf as described in the Networking section).

Note: you need to be careful if host services are configured to run on all interfaces as this will include the jail interface - you should specify the interface they run on in rc.conf (or other config files)

```
# bastille rdr dev1 tcp 2001 22
[jail1]:
IPv4 tcp/2001:22  on em0

# bastille rdr dev1 list
rdr on em0 inet proto tcp from any to any port = 2001 -> 10.17.89.1 port 22

# bastille rdr dev1 udp 2053 53
[jail1]:
IPv4 udp/2053:53 on em0

# bastille rdr dev1 list
rdr pass on em0 inet proto tcp from any to any port = 2001 -> 10.17.89.1 port 22
rdr pass on em0 inet proto udp from any to any port = 2053 -> 10.17.89.1 port 53

# bastille rdr dev1 clear
nat cleared
```

The `rdr` command includes 4 additional options:

```
-d | --destination IP            Limit rdr to a destination IP. Useful if you have␣
↪multiple IPs on one interface.
-i | --interface   IF,IF         Specify interface(s) to apply rule to. Comman separated.
-s | --source      IP|table      Limit rdr to a source IP or table.
-t | --type        ipv4|ipv6     Specify IP type. Must be used if -s or -d are used.␣
↪Defaults to both.
-x | --debug                     Enable debug mode.
```

```
# bastille rdr -i vtnet0 dev1 udp 8000 80
[jail1]:
IPv4 tcp/8000:80 on vtnet0

# bastille rdr -s 192.168.0.1 dev1 tcp 8080 81
[jail1]:
IPv4 tcp/8080:81 on em0

# bastille rdr -d 192.168.0.84 dev1 tcp 8082 82
[jail1]:
IPv4 tcp/8082:82 on em0

# bastille rdr -i vtnet0 -d 192.168.0.45 dev1 tcp 9000 9000
[jail1]:
IPv4 tcp/9000:9000 on vtnet0

# bastille rdr dev1 list
```

(continues on next page)

(continued from previous page)

```
rdr pass on vtnet0 inet proto udp from any to any port = 2001 -> 10.17.89.1 port 22
rdr pass on em0 inet proto tcp from 192.168.0.1 to any port = 8080 -> 10.17.89.1 port 81
rdr pass on em0 inet proto tcp from any to 192.168.0.84 port = 8082 -> 10.17.89.1 port 82
rdr pass on vtnet0 inet proto tcp from any to 192.168.0.45 port = 9000 -> 10.17.89.1␣
↪port 9000
```

The options can be used together, as seen above.

If you have multiple interfaces assigned to your jail, `bastille rdr` will only redirect using the default one.

It is also possible to specify a pf table as the source, providing it exists. Simply use the table name instead of an IP address or subnet.

```
# bastille rdr --help
Usage: bastille rdr [option(s)] TARGET tcp|udp HOST_PORT JAIL_PORT [log LOG_OPTIONS]
                                TARGET clear|reset|list

    Options:

    -d | --destination IP            Limit rdr to a destination IP.
    -i | --interface   IF,IF         Specify interface(s) to apply rule to. Comma-
↪separated.
    -s | --source      IP|TABLE      Limit rdr to a source IP or table.
    -t | --type        ipv4|ipv6     Specify IP type. Must be used if '-s' or '-d' are␣
↪used. Defaults to both.
    -x | --debug                     Enable debug mode.
```

6.25 rename

```
ishmael ~ # bastille rename azkaban arkham
```

```
ishmael ~ # bastille rename help
Usage: bastille rename [option(s)] TARGET NEW_NAME

    Options:

    -a | --auto      Auto mode. Start/stop jail(s) if required.
    -x | --debug     Enable debug mode.
```

6.26 restart

Bastille will attempt to stop, then start the targetted jail(s). If a jail is not running, Bastille will still start it. To avoid this, run the restart command with `-i|--ignore` to skip any stopped jail(s).

```
ishmael ~ # bastille restart folsom
[folsom]:
folsom: removed
[folsom]:
folsom: created
```

```
ishmael ~ # bastille restart help
Usage: bastille start [option(s)] TARGET

    Options:

    -b | --boot            Respect jail boot setting.
    -d | --delay VALUE     Time (seconds) to wait after starting each jail.
    -i | --ignore          Ignore stopped jails (do not start if stopped).
    -v | --verbose         Enable verbose mode.
    -x | --debug           Enable debug mode.
```

6.27 service

The `service` sub-command allows for managing services within jails. This allows you to start, stop, restart, and otherwise interact with services running inside the jail(s).

```
ishmael ~ # bastille service web01 'nginx start'
ishmael ~ # bastille service db01 'mysql-server restart'
ishmael ~ # bastille service proxy 'nginx configtest'
ishmael ~ # bastille service proxy 'nginx enable'
ishmael ~ # bastille service proxy 'nginx disable'
ishmael ~ # bastille service proxy 'nginx delete'
```

```
ishmael ~ # bastille service help
Usage: bastille service [option(s)] TARGET SERVICE ARGS

    Options:

    -a | --auto      Auto mode. Start/stop jail(s) if required.
    -x | --debug     Enable debug mode.
```

6.28 setup

The `setup` sub-command attempts to automatically configure a host system for Bastille jails. This allows you to configure networking, firewall, storage, and some additional options for a Bastille host with one command.

6.28.1 Options

Below is a list of available options that can be used with the `setup` command.

The `bridge` options will attempt to configure a bridge interface for use with bridged VNET (`-B`) jails.

The `linux` options will attempt to configure your system to run Linux (`-L|--linux`) jails. This will load some required kernel modules, and add the to `/boot/loader.conf`.

The `loopback` option will configure a loopback interface called `bastille0` that will be used as a default when not specifying an interface with the `create` command.

The `netgraph` option will attempt to configure your system to use `netgraph` as the network mode as opposed to the standard `if_bridge` mode.

The `pf|firewall` option will configure the pf firewall by enabling the service and creating the default `pf.conf` file. Once this is done, you can use the `rdr` command to forward traffic into a jail.

The `shared` option will configure the interface you choose to also be used as the default when not specifying an interface with the `create` command.

The `storage` option will attempt to configure a pool and dataset for Bastille, but only if ZFS in enabled on your system. Otherwise it will use UFS.

The `vnet` option will configure your system for use with VNET (`-V`) jails.

6.28.2 Limitations

The `loopback` option is the default, and is enough for most use cases. It is simply an `lo` interface that jails will get linked to on creation. It is not attached to any specific interface. This is the simplest networking option. The `loopback` and `shared` options are only for cases where the `interface` is not specified during the `create` command. If an interface is specified, these options have no effect. Instead, the specified interface will be used.

Please note. You CANNOT run both a loopback and a shared interface with Bastille. Only one should be configured. If you configure one, it will disable the other. The `shared` option is for cases where you want an actual interface to use with Bastille as opposed to a loopback. Jails will be linked to the shared interface on creation.

Running `bastille setup` without any options will attempt to auto-configure the `loopback`, `firewall` and `storage` options.

```
ishmael ~ # bastille setup -h
Usage: bastille setup [option(s)]␣
↪[bridge|linux|loopback|netgraph|firewall|shared|storage|vnet]

    Options:

    -y | --yes       Do not prompt. Assume always yes.
    -x | --debug     Enable debug mode.
```

6.29 start

```
ishmael ~ # bastille start folsom
[folsom]:
folsom: created
```

```
ishmael ~ # bastille start help
Usage: bastille start [option(s)] TARGET

    Options:

    -b | --boot            Respect jail boot setting.
    -d | --delay VALUE     Time (seconds) to wait after starting each jail.
    -v | --verbose         Enable verbose mode.
    -x | --debug           Enable debug mode.
```

6.30 stop

```
ishmael ~ # bastille stop folsom
[folsom]:
folsom: removed
```

```
ishmael ~ # bastille stop help
Usage: bastille stop [option(s)] TARGET

    Options:

    -v | --verbose     Enable verbose mode.
    -x | --debug       Enable debug mode.
```

6.31 sysrc

The `sysrc` sub-command allows for safely editing system configuration files. In jail terms, this allows us to toggle on/off services and options at startup.

```
ishmael ~ # bastille sysrc nginx nginx_enable="YES"
[nginx]:
nginx_enable: NO -> YES
```

See `man sysrc(8)` for more info.

```
ishmael ~ # bastille sysrc help
Usage: bastille sysrc [option(s)] TARGET ARGS

    Options:

    -a | --auto      Auto mode. Start/stop jail(s) if required.
    -x | --debug     Enable debug mode.
```

6.32 tags

```
ishmael ~ # bastille tags help                  ## display tags help
ishmael ~ # bastille tags TARGET add tag1,tag2  ## add the tags "tag1" and "tag2" to␣
↪TARGET
ishmael ~ # bastille tags TARGET delete tag2    ## delete tag "tag2" from TARGET
ishmael ~ # bastille tags TARGET list           ## list tags assigned to TARGET
ishmael ~ # bastille tags ALL list              ## list tags from ALL containers
```

```
ishmael ~ # bastille tags help
Usage: bastille tags [option(s)] TARGET add|delete TAG1,TAG2
                                 TARGET list [TAG]

    Options:

    -x | --debug     Enable debug mode.
```

6.33 template

```
ishmael ~ # bastille template azkaban project/template
```

Templates should be structured in `project/template/Bastillefile` format, and placed in the template directory, which defaults to `/usr/local/bastille/templates`. The Bastillefile should contain the template hooks. See the

chapter called Template for a list of supported hooks.

The TEMPLATE arg should be called with the `project/template` format.

```
ishmael ~ # bastille template help
Usage: bastille template [option(s)] TARGET|convert TEMPLATE

    Options:

    -a | --auto      Auto mode. Start/stop jail(s) if required.
    -x | --debug     Enable debug mode.
```

6.34 top

```
last pid: 96267;  load averages:  0.16,  0.17,  0.11   up 10+22:37:33  21:59:07
8 processes:   1 running, 7 sleeping
CPU:  0.0% user,  0.0% nice,  0.0% system,  0.0% interrupt,  100% idle
Mem: 6768K Active, 323M Inact, 15G Wired, 7891M Free
ARC: 8391M Total, 4004M MFU, 3376M MRU, 64K Anon, 68M Header, 943M Other
     5937M Compressed, 11G Uncompressed, 1.84:1 Ratio
Swap: 2048M Total, 2048M Free

  PID USERNAME    THR PRI NICE   SIZE    RES STATE   C   TIME    WCPU COMMAND
96267 root          1  20    0  7916K  3192K CPU1    1   0:00   0.01% top
34907 nobody        1  20    0 11236K  7552K kqread  6   0:00   0.00% nginx
36263 nobody        1  20    0 11236K  7552K kqread  4   0:00   0.00% nginx
36867 nobody        1  20    0 11236K  7556K kqread  2   0:00   0.00% nginx
35024 nobody        1  20    0 11236K  7540K kqread  6   0:00   0.00% nginx
40049 root          1  20    0  6464K  2372K nanslp  4   0:00   0.00% cron
10201 root          1  20    0  6412K  2368K select  5   0:00   0.00% syslogd
34902 root          1  52    0 11236K  6920K pause   4   0:00   0.00% nginx
```

```
ishmael ~ # bastille top help
Usage: bastille top [options(s)] TARGET

    Options:

    -a | --auto      Auto mode. Start/stop jail(s) if required.
    -x | --debug     Enable debug mode.
```

6.35 umount

```
ishmael ~ # bastille umount azkaban /media/foo
[azkaban]:
Unmounted: /usr/local/bastille/jails/jail4/root/media/foo
```

(continues on next page)

(continued from previous page)

```
ishmael ~ # bastille umount azkaban /mnt/etc/rc.conf
[azkaban]:
Unmounted: /usr/local/bastille/jails/jail4/root/mnt/etc/rc.conf
```

Syntax requires only the jail path to unmount.

```
Usage: bastille umount TARGET JAIL_PATH
```

If the directory you are unmounting has spaces, make sure to escape them with a backslash , and enclose the mount point in quotes “”.

```
ishmael ~ # bastille umount azkaban "/media/foo\ with\ spaces"
[azkaban]:
Unmounted: /usr/local/bastille/jails/jail4/root/media/foo with spaces
```

```
ishmael ~ # bastille umount help
Usage: bastille umount [option(s)] TARGET JAIL_PATH

    Options:

    -a | --auto      Auto mode. Start/stop jail(s) if required.
    -x | --debug     Enable debug mode.
```

6.36 update

The `update` command targets a release or a thick jail. Because thin jails are based on a release, when the release is updated all the thin jails are automatically updated as well.

If no updates are available, a message will be shown:

```
ishmael ~ # bastille update 11.4-RELEASE
Looking up update.FreeBSD.org mirrors... 2 mirrors found.
Fetching metadata signature for 11.4-RELEASE from update4.freebsd.org... done.
Fetching metadata index... done.
Inspecting system... done.
Preparing to download files... done.

No updates needed to update system to 11.4-RELEASE-p4.
No updates are available to install.
```

The older the release or jail, however, the more updates will be available:

```
ishmael ~ # bastille update 13.2-RELEASE
Looking up update.FreeBSD.org mirrors... 2 mirrors found.
Fetching metadata signature for 13.2-RELEASE from update1.freebsd.org... done.
Fetching metadata index... done.
Fetching 2 metadata patches.. done.
Applying metadata patches... done.
Fetching 2 metadata files... done.
Inspecting system... done.
Preparing to download files... done.
```

(continues on next page)

(continued from previous page)

```
The following files will be added as part of updating to 13.2-RELEASE-p4:
...[snip]...
```

To be safe, you may want to restart any jails that have been updated live.

If the jail is a thin jail, an error will be shown. If it is a thick jail, it will be updated just like the release shown above.

```
ishmael ~ # bastille update help
Usage: bastille update [option(s)] TARGET

    Options:

    -a | --auto      Auto mode. Start/stop jail(s) if required.
    -f | --force     Force update a release (FreeBSD legacy releases).
    -x | --debug     Enable debug mode.
```

6.37 upgrade

The `upgrade` command targets a thick or thin jail. Thin jails will be updated by changing the release mount point that it is based on. Thick jails will be upgraded normally.

```
ishmael ~ # bastille upgrade help
Usage: bastille upgrade [option(s)] TARGET NEW_RELEASE
                                    TARGET install

    Options:

    -a | --auto      Auto mode. Start/stop jail(s) if required.
    -f | --force     Force upgrade a release (FreeBSD legacy releases).
    -x | --debug     Enable debug mode.
```

6.38 verify

This command scans a bootstrapped release or template and validates that everything looks in order. This is not a 100% comprehensive check, but it compares the release or template against a "known good" index.

If you see errors or issues here, consider deleting and re-bootstrapping the release or template .

```
ishmael ~ # bastille verify 11.2-RELEASE
Looking up update.FreeBSD.org mirrors... 2 mirrors found.
Fetching metadata signature for 11.2-RELEASE from update1.freebsd.org... done.
Fetching metadata index... done.
Fetching 1 metadata patches. done.
Applying metadata patches... done.
Fetching 1 metadata files... done.
Inspecting system... done.

ishmael ~ # bastille verify bastillebsd-templates/jellyfin
Detected Bastillefile hook.
[Bastillefile]:
CMD mkdir -p /usr/local/etc/pkg/repos
```

(continues on next page)

(continued from previous page)

```
CMD echo 'FreeBSD: { url: "pkg+http://pkg.FreeBSD.org/${ABI}/latest" }' >
/usr/local/etc/pkg/repos/FreeBSD.conf
CONFIG set allow.mlock=1;
CONFIG set ip6=inherit;
RESTART
PKG jellyfin
SYSRC jellyfin_enable=TRUE
SERVICE jellyfin start
Template ready to use.
```

```
ishmael ~ # bastille verify help
Usage: bastille verify [option(s)] RELEASE|TEMPLATE

    Options:

    -x | --debug     Enable debug mode.
```

6.39 zfs

Manage ZFS properties, create, destroy and rollback snapshots, jail and unjail datasets (ZFS only), and check ZFS usage for targeted jail(s).

6.39.1 Snapshot Management

Bastille has the ability to create, destroy, and rollback snapshots when using ZFS. To create a snapshot, run `bastille zfs TARGET snapshot`. This will create a snapshot with the default `bastille_TARGET_DATE` naming scheme. You can also specify a TAG to use as the naming scheme, such as `bastille zfs TARGET snapshot mytag`. Bastille will then create the snapshot with `@mytag` as the snapshot name.

Rolling back a snapshot follows the same syntax. If no TAG is supplied, Bastille will attempt to use the most recent snapshot following the default naming scheme above. To rollback a snapshot with a custom tag, run `bastille zfs TARGET rollback` or `bastille zfs TARGET rollback mytag`.

To destroy a snaphot however, you must supply a TAG. To destroy a snapshot, run `bastille zfs TARGET destroy mytag`.

```
ishmael ~ # bastille zfs help
Usage: bastille zfs [option(s)] TARGET snapshot|destroy|rollback [TAG]"
                                        df|usage"
                                        get|set KEY=VALUE"
                                        jail pool/dataset /jail/path"
                                        unjail pool/dataset"

    Options:

    -a | --auto        Auto mode. Start/stop jail(s) if required.
    -v | --verbose     Enable verbose mode.
    -x | --debug       Enable debug mode.
```

CHAPTER

SEVEN

USAGE

```
  ishmael ~ # bastille help
  Bastille is an open-source system for automating deployment and management of
  containerized applications on FreeBSD.

Usage:
  bastille [option(s)] command [option(s)] TARGET ARGS

Available Commands:
  bootstrap   Bootstrap a release or template(s).
  clone       Clone an existing jail.
  cmd         Execute command(s) inside jail(s).
  config      Get, set, add or remove properties from jail(s).
  console     Console into a jail.
  convert     Convert a jail from thin to thick; convert a jail to a custom release.
  cp          Copy file(s)/directorie(s) from host to jail(s).
  create      Create a jail.
  destroy     Destroy jail(s) or release(s).
  edit        Edit jail configuration files (advanced).
  etcupdate   Update /etc for jail(s).
  export      Export a jail.
  help        Help for any command.
  htop        Interactive process viewer (requires htop).
  import      Import a jail.
  jcp         Copy file(s)/directorie(s) from jail to jail(s).
  limits      Apply resources limits to jail(s). See rctl(8) and cpuset(1).
  list        List jails, releases, templates and more...
  migrate     Migrate jail(s) to a remote system.
  monitor     Monitor and attempt to restart jail service(s).
  mount       Mount file(s)/directorie(s) inside jail(s).
  network     Add or remove interface(s) from jail(s).
  pkg         Manage packages inside jail(s). See pkg(8).
  rcp         Copy file(s)/directorie(s) from jail to host.
  rdr         Redirect host port to jail port.
  rename      Rename a jail.
  restart     Restart a jail.
  service     Manage services within jail(s).
  setup       Auto-configure network, firewall, storage and more...
  start       Start stopped jail(s).
  stop        Stop running jail(s).
  sysrc       Edit rc files inside jail(s).
```

(continues on next page)

(continued from previous page)

```
  tags        Add or remove tags to jail(s).
  template    Apply templates to jail(s).
  top         Process viewer. See top(1).
  umount      Unmount file(s)/directorie(s) from jail(s).
  update      Update a jail or release.
  upgrade     Upgrade a jail to new release.
  verify      Compare release against a "known good" index.
  zfs         Manage ZFS options/attributes for jail(s).

Use "bastille -v|--version" for version information.
Use "bastille command -h|--help" for more information about a command.
Use "bastille -c|--config FILE command" to specify a non-default config file.
```

CHAPTER

EIGHT

NETWORKING

Bastille is very flexible with its networking options. Below are the supported networking modes, how they work, and some tips on where you might want to use each one.

Bastille also supports VLANs to some extent. See the VLAN section below.

8.1 Jail Network Modes

Bastille tries to be flexible in the different network modes it supports. Below is a breakdown of each network mode, what each one does, as well as some suggestions as to where you might want to use each one.

8.1.1 VNET

- For VNET jails (`-V`) Bastille will create a bridge interface and attach your jail to it. It will be called `em0bridge` or whatever your interface is called. This will be used for the host/jail epairs. Bastille will create/destroy these epairs as the jail is started/stopped.
- This mode works best if you want your jail to be in your local network, acting as a physical device with its own MAC address and IP.

8.1.2 Bridged VNET

- For bridged VNET jails (`-B`) you must manually create a bridge interface to attach your jail to. Bastille will then create and attach the host/jail epairs to this interface when the jail starts, and remove themwhen it stops.
- This mode is identical to *VNET* above, with one exception. The interface it is attached to is a manually created bridge, as opposed to a regular interface that is used with *VNET* above.

8.1.3 Alias/Shared Interface

- For classic/standard jails that use an IP that is accessible within your local subnet (alias mode) Bastille will add the IP to the specified interface as an alias.
- This mode is best used if you have one interface, and don't want the jail to have its own MAC address. The jail IP will simply be added to the specified interface as an additional IP, and will inherit the rest of the interface.
- Note that this mode does not function as the two *VNET* modes above, but still allows the jail to have an IP address inside your local network.

8.1.4 NAT/Loopback Interface

- For classic/standard jails that use an IP not reachable in your local subnet, Bastille will add the IP to the specified interface as an alias, and additionally, add it to the pf firewall table (if available) to allow the jail outbound access. If you do not specify an interface, Bastille will assume you have run the `bastille setup` command and will

attempt to use `bastille0` (which is created using the setup command) as its interface. If you have not run `bastille setup` and do not specify an interface, Bastille will error.

- This mode works best if you want your jail to be in its own private network. Bastille will dynamically add each jail IP to the firewall table to ensure network connectivity.
- This mode is similar to the Alias/Shared Interface mode, except that it is not limited to IP addresses within your local network.

8.1.5 Inherit

- For classic/standard jails that are set to `inherit` or `ip_hostname`, bastille will simply set `ip4` to `inherit` inside the jail config. The jail will then function according the jail(8) documentation.
- This mode makes the jail inherit the entire network stack of the host.

8.1.6 IP Hostname

- For classic/standard jails that are set to `ip_hostname`, bastille will simply set `ip4` to `ip_hostname` inside the jail config. The jail will then function according the jail(8) documentation.
- This is an advanced parameter. See the official FreeBSD jail(8) documentation for details.

You cannot use `-V|--vnet` with any interface that is already a member of another bridge. For example, if you create a bridge, and assign `vtnet0` as a member, you will not be able to use `vtnet0` with `-V|--vnet`.

8.2 IP Address Options

8.2.1 IPv4 Network

Bastille includes a number of IP options for IPv4 networking.

```
bastille create alcatraz 13.2-RELEASE 192.168.1.50/24 vtnet0
```

The IP address specified above can be any of the following options.

- An IP in your local subnet should be chosen if you create your jail using `-V`, `-B` or `-P` (VNET jail).

  Note: It is mandatory to add the subnet mask (/24 or whaterver your subnet is) to the IP for any types of VNET jail. See below…
- DHCP, SYNCDHCP, or 0.0.0.0 will configure your jail to use DHCP to obtain an address from your router. This should only be used with VNET jails.
- Any IP address inside the RFC1918 range if you are not using a VNET jail. Bastille will automatically add this IP to the firewall table to allow outbound access. It you want traffic to be forwarded into the jail, you can use the `bastille rdr` command.
- Any IP in your local subnet without any VNET options will add the IP as an alias to the selected interface, which will simply end up sharing the interface. If the IP is in your local subnet, you will not need the `bastille rdr` command. Traffic will pass in and out just as in a VNET jail.
- Setting the IP to `inherit` will make the jail inherit the entire host network stack.
- Setting the IP to `ip_hostname` will add all the IPs that the hostname resolves to. This is an advanced option and should only be used if you know what you are doing.

Standard (non-VNET) jails support specifying an IP without the subnet (/24 or whatever yours is), but for VNET jails it is mandatory. If none is supplied, it will default to /24. This is because FreeBSD does not support adding an IP to an interface without a subnet.

8.2.2 IPv6 Network

Bastille also supports IPv6. Instead of an IPv4 address, you can specify an IPv6 address when creating a jail to use IPv6.

```
bastille create alcatraz 13.2-RELEASE 2001:19f0:6c01:114c:0:100/64 vtnet0
```

The IP address specified above can be any of the following options.

- A valid IPv6 address including the subnet. If not subnet is given, it will defalut to /64.
- SLAAC will configure your jail to use router advertisement to obtain an address from your router. This should only be used with VNET jails.

8.2.3 Dual Stack Network

It is also possible to use both IPv4 and IPv6 by quoting an IPv4 and IPv6 addresses together as seen in the following examples.

```
bastille create alcatraz 14.3-RELEASE "192.168.1.50/24 2001:19f0:6c01:114c:0:100/64"␣
↪vtnet0
```

```
bastille create alcatraz 14.3-RELEASE "DHCP SLAAC" vtnet0
```

Note: For the `inherit` and `ip_hostname` options, you can also specify `-D|--dual` to use both IPv4 and IPv6 inside the jail. Otherwise, for dual stack networking, simply supply both IPv4 and IPv6 addresses as seen above.

8.3 Networking Limitations

8.3.1 VNET Jail Interface Names

- FreeBSD has certain limitations when it comes to interface names. One of these is that interface names cannot be longer than 15 characters. Because of this, Bastille uses a generic name for any epairs created whose corresponding jail name exceeds the maximum length. See below...

  `e0a_jailname` and `e0b_jailname` are the default epair interfaces for every jail. The `e0a` side is on the host, while the `e0b` is in the jail. Due to the above mentioned limitations, Bastille will name any epairs whose jail names exceed the maximum length, to `e0b_bastilleX` and `e0b_bastilleX` with the `X` starting at `1` and incrementing by 1 for each new epair. So, `mylongjailname` will be `e0a_bastille2` and `e0b_bastille2`.

8.3.2 Netgraph and Proxmox VE

- When running a FreeBSD VM on Proxmox VE, you might encounter crashes when using Netraph. This bug is being tracked at https://bugs.freebsd.org/bugzilla/show_bug.cgi?id=238326

  One workaround is to add the following line to the `jail.conf` file of the affected jail(s).

```
exec.prestop += "jng shutdown JAILNAME";
```

8.4 Network Scenarios

8.4.1 SOHO (Small Office/Home Office)

This scenario works best when you have just one computer, or a home or small office network that is separated from the rest of the internet by a router. So you are free to use private IP addresses.

In this environment, we can create the container, give it a unique private ip address within our local subnet, and attach its ip address to our primary interface.

```
bastille create alcatraz 13.2-RELEASE 192.168.1.50 em0
```

You may have to change em0

When the `alcatraz` container is started it will add `192.168.1.50` as an IP alias to the `em0` interface. It will then simply be another member of the hosts network. Other networked systems (firewall permitting) should be able to reach services at that address.

This method is the simplest. All you need to know is the name of your network interface and a free IP on your local network.

We can also run `bastille setup shared` to configure our primary interface as a default interface for Bastille to use. Once we have run the command and chosen our interface, it will not be necessary to specify an interface in our create command.

```
bastille create alcatraz 13.2-RELEASE 192.168.1.50
```

This will automatically use the interface we selected during the setup command.

Note that we cannot use the `shared` option together with the `loopback` option. Configuring one using the `bastille setup` command will disable the other.

8.4.2 Shared Interface on IPV6 network (vultr.com)

Some ISP's, such as Vultr, give you a single ipv4 address, and a large block of ipv6 addresses. You can then assign a unique ipv6 address to each Bastille Container.

On a virtual machine such as vultr.com the virtual interface may be `vtnet0`. So we issue the command:

```
bastille create alcatraz 13.2-RELEASE 2001:19f0:6c01:114c::100 vtnet0
```

We could also write the ipv6 address as 2001:19f0:6c01:114c:0:100

The tricky part are the ipv6 addresses. IPV6 is a string of 8 4 digit hexadecimal characters. At vultr they said:

Your server was assigned the following six section subnet:

2001:19f0:6c01:114c::/64

The vultr ipv6 subnet calculator is helpful in making sense of that ipv6 address.

We could have also written that IPV6 address as 2001:19f0:6c01:114c:0:0

Where the /64 basicaly means that the first 64 bits of the address (4x4 character hexadecimal) values define the network, and the remaining characters, we can assign as we want to the Bastille Container. In the actual bastille create command given above, it was defined to be 100. But we also have to tell the host operating system that we are now using this address. This is done on freebsd with the following command

```
ifconfig_vtnet0_alias0="inet6 2001:19f0:6c01:114c::100 prefixlen 64"
```

At that point your container can talk to the world, and the world can ping your container. Of course when you reboot the machine, that command will be forgotten. To make it permanent, prefix the same command with `sysrc`

Just remember you cannot ping out from the container. Instead, install and use `wget/curl/fetch` to test the connectivity.

8.4.3 VNET (Virtual Network)

(Added in 0.6.x) VNET is supported on FreeBSD 12+ only.

Virtual Network (VNET) creates a private network interface for a container. This includes a unique hardware address. This is required for VPN, DHCP, and similar containers.

To create a VNET based container use the `-V|--vnet` option, an IP/netmask and external interface.

```
bastille create -V azkaban 13.2-RELEASE 192.168.1.50/24 em0
```

Bastille will automagically create the bridge interface and connect / disconnect containers as they are started and stopped. A new interface will be created on the host matching the pattern `interface0bridge`. In the example here, `em0bridge`.

The `em0` interface will be attached to the bridge along with the unique container interfaces as they are started and stopped. These interface names match the pattern `eXb_bastilleX`. Internally to the containers these interfaces are presented as `vnet0`.

If you do not specify a subnet mask, you might have issues with jail to jail networking, especially VLAN to VLAN. We recommend always adding a subnet to VNET jail IPs when creating them to avoid these issues.

VNET also requires a custom devfs ruleset. Create the file as needed on the host system:

```
## /etc/devfs.rules (NOT .conf)

[bastille_vnet=13]
add include $devfsrules_hide_all
add include $devfsrules_unhide_basic
add include $devfsrules_unhide_login
add include $devfsrules_jail
add include $devfsrules_jail_vnet
add path 'bpf*' unhide
```

Lastly, you may want to consider these three `sysctl` values:

```
net.link.bridge.pfil_bridge=0
net.link.bridge.pfil_onlyip=0
net.link.bridge.pfil_member=0
```

Below is the definition of what these three parameters are used for and mean:

net.link.bridge.pfil_onlyip Controls the handling of non-IP packets
: which are not passed to pfil(9). Set to 1 to only allow IP packets to pass (subject to firewall rules), set to 0 to uncondi- tionally pass all non-IP Ethernet frames.

net.link.bridge.pfil_member Set to 1 to enable filtering on the incom-
: ing and outgoing member interfaces, set to 0 to disable it.

net.link.bridge.pfil_bridge Set to 1 to enable filtering on the bridge
: interface, set to 0 to disable it.

8.4.4 Bridged VNET (Virtual Network)

To create a VNET based container and attach it to an external, already existing bridge, use the `-B` option, an IP/netmask and external bridge.

```
bastille create -B azkaban 13.2-RELEASE 192.168.1.50/24 bridge0
```

Bastille will automagically create the needed interface(s), attach it to the specified bridge and connect/disconnect containers as they are started and stopped. The bridge needs to be created/enabled before creating and starting the jail.

Below are the steps to creating a bridge for this purpose.

The first thing you have to do is to create a bridge interface on your system. This is done with the ifconfig command and will create a bridged interface named bridge0:

```
ifconfig bridge create
```

Then you need to add your system's network interface to the bridge and bring it up (substitute your interface for em0).

```
ifconfig bridge0 addm em0 up
```

Optionally you can rename the interface if you wish to make it obvious that it is for bastille:

```
ifconfig bridge0 name bastille0bridge
```

To create a bridged container you use the `-B` option, an IP or DHCP, and the bridge interface.

```
bastille create -B folsom 14.2-RELEASE DHCP bastille0bridge
```

All the epairs and networking other than the manually created bridge will be created for you automagically. Now if you want this to persist after a reboot then you need to add some lines to your `/etc/rc.conf` file. Add the following lines, again, obviously change em0 to whatever your network interface on your system is.

```
cloned_interfaces="bridge0"
ifconfig_bridge0_name="bastille0bridge"
ifconfig_bastille0bridge="addm vtnet0 up"
```

8.5 VLAN Configuration

8.5.1 Jail VLAN Tagging

Bastille supports VLANs to some extent when creating jails. When creating a jail, use the `--vlan ID` options to specify a VLAN ID for your jail. This will set the proper variables inside the jails *rc.conf* to add the jail to the specified VLAN. The jail will then take care of tagging the traffic. Do not use `-v|--vlan` if you have already configured the host interface to tag the traffic. See limitations below.

When using this method, the interface being assigned must be a trunk interface. This means that it passes all traffic, leaving any VLAN tags as they are.

8.5.2 Host VLAN Tagging

Another method is to configure a host interface to tag the traffic. This way, the jail doesn't have to worry about it.

You can only use `-B|--bridge` with host VLAN interfaces, due to the limitation mentioned below. With this method we create the bridge interfaces in `rc.conf` and configure them to tag the traffic by VLAD ID.

Below is an `rc.conf` snippet that was provided by a user who has such a configuration.

```
# rename ethernet interfaces (optional)
ifconfig_igb1_name="eth1"
ifconfig_eth1_descr="vm/jail ethernet interface"

# setup vlans
```

(continues on next page)

(continued from previous page)

```
vlans_eth1="10 20 30"

# setup bridges
cloned_interfaces="bridge10 bridge20 bridge30"
ifconfig_bridge10_name="eth1.10bridge"
ifconfig_bridge20_name="eth1.20bridge"
ifconfig_bridge30_name="eth1.30bridge"
ifconfig_eth1_10bridge="addm eth1.10 up"
ifconfig_eth1_20bridge="addm eth1.20 up"
ifconfig_eth1_30bridge="addm eth1.30 up"

# bring interfaces up
ifconfig_eth1="up"
ifconfig_eth1_10="up"
ifconfig_eth1_20="up"
ifconfig_eth1_30="up"
```

Notice that the interfaces are bridge interfaces, and can be used with `-B|--bridge` without issue.

8.5.3 VLAN Limitations

- You cannot use the `-V|--vnet` options with interfaces that have dots (.) in the name, which is the standard way of naming a VLAN interface. This is due to the limitations of the JIB script that Bastille uses to manage VNET jails.
- Do not attempt to configure both the host and the jail to tag VLAN traffic. If you use the host method, do not use `-v|--vlan` when creating the jail. Doing so will prevent the jail from having network access.

Tip: Don't forget to set you gateway and nameserver is applicable using `-g|--gateway` and `-n|--nameserver`.

8.6 Regarding Routes

Bastille will attempt to auto-detect the default route from the host system and assign it to the VNET container. This auto-detection may not always be accurate for your needs for the particular container. In this case you'll need to add a default route manually or define the preferred default route in the `bastille.conf`.

```
bastille sysrc TARGET defaultrouter=aa.bb.cc.dd
bastille service TARGET routing restart
```

To define a default route / gateway for all VNET containers define the value in `bastille.conf`:

```
bastille_network_gateway=aa.bb.cc.dd
```

This config change will apply the defined gateway to any new containers. Existing containers will need to be manually updated.

8.7 Public Network

In this section we describe how to network containers in a public network such as a cloud hosting provider who only provides you with a single ip address. (AWS, Digital Ocean, etc) (The exception is vultr.com, which does provide you with lots of IPV6 addresses and does a great job supporting FreeBSD!)

So if you only have a single IP address and if you want to create multiple containers and assign them all unique IP addresses, you'll need to create a new network.

8.8 Netgraph

Bastille supports netgraph as an VNET management tool, thanks to the *jng* script. To enable netgraph, run *bastille setup netgraph*. This will load and persist the required kernel modules. Once netgraph is configured, any VNET jails you create will be managed with netgraph.

Note that you should only enable netgraph on a new system. Bastille is set up to use either *netgraph* or *if_bridge* as the VNET management, and uses *if_bridge* as the default, as it always has. The *netgraph* option is new, and should only be used with new systems.

This value is set with the *bastille_network_vnet_type* option inside the config file.

8.8.1 loopback (bastille0)

What we recommend is creating a cloned loopback interface (`bastille0`) and assigning all the containers private (rfc1918) addresses on that interface. The setup I develop on and use Bastille day-to-day uses the `10.0.0.0/8` address range. I have the ability to use whatever address I want within that range because I've created my own private network. The host system then acts as the firewall, permitting and denying traffic as needed.

I find this setup the most flexible across all types of networks. It can be used in public and private networks just the same and it allows me to keep containers off the network until I allow access.

Having said all that here are instructions I used to configure the network with a private loopback interface and system firewall. The system firewall NATs traffic out of containers and can selectively redirect traffic into containers based on connection ports (ie; 80, 443, etc.)

To set up the loopback address automatically, we can simply run `bastille setup`. This will configure the storage, pf firewall, and loopback addresses for us. To set these up individually, we can run `bastille setup storage`, `bastille setup firewall`, and `bastille setup loopback` respectively.

Alternatively, you can do it all manually, as shown below.

First, create the loopback interface:

```
ishmael ~ # sysrc cloned_interfaces+=lo1
ishmael ~ # sysrc ifconfig_lo1_name="bastille0"
ishmael ~ # service netif cloneup
```

Second, enable the firewall:

```
ishmael ~ # sysrc pf_enable="YES"
```

Create the firewall rules:

8.8.2 /etc/pf.conf

```
ext_if="vtnet0"

set block-policy return
scrub in on $ext_if all fragment reassemble
set skip on lo

table <jails> persist
nat on $ext_if from <jails> to any -> ($ext_if:0)
rdr-anchor "rdr/*"

block in all
```

(continues on next page)

(continued from previous page)

```
pass out quick keep state
antispoof for $ext_if inet
pass in proto tcp from any to any port ssh flags S/SA modulate state
```

- Make sure to change the `ext_if` variable to match your host system interface.
- Make sure to include the last line (`port ssh`) or you'll end up locked out.

Note: if you have an existing firewall, the key lines for in/out traffic to containers are:

```
nat on $ext_if from <jails> to any -> ($ext_if:0)
```

The `nat` routes traffic from the loopback interface to the external interface for outbound access.

```
rdr-anchor "rdr/*"
```

The `rdr-anchor "rdr/*"` enables dynamic rdr rules to be setup using the `bastille rdr` command at runtime - eg.

```
bastille rdr TARGET tcp 2001 22 # Redirects tcp port 2001 on host to 22 on jail
bastille rdr TARGET udp 2053 53 # Same for udp
bastille rdr TARGET list        # List dynamic rdr rules
bastille rdr TARGET clear       # Clear dynamic rdr rules
```

Note that if you are redirecting ports where the host is also listening (eg. ssh) you should make sure that the host service is not listening on the cloned interface - eg. for ssh set sshd_flags in rc.conf

```
sshd_flags="-o ListenAddress=<host-address>"
```

Finally, start up the firewall:

```
ishmael ~ # service pf restart
```

At this point you'll likely be disconnected from the host. Reconnect the ssh session and continue.

This step only needs to be done once in order to prepare the host.

Note that we cannot use the `loopback` option together with the `shared` option. Configuring one using the `bastille setup` command will disable the other.

8.9 local_unbound

If you are running "local_unbound" on your server, you will probably have issues with DNS resolution.

To resolve this, add the following configuration to local_unbound:

```
server:
interface: 0.0.0.0
access-control: 192.168.0.0/16 allow
access-control: 10.17.90.0/24 allow
```

Also, change the nameserver to the servers IP instead of 127.0.0.1 inside /etc/rc.conf

Adjust the above "access-control" strings to fit your network.

CHAPTER

NINE

BASTILLE VNET ON GCP

Bastille VNET runs on GCP with a few small tweaks. In summary, they are:

- change MTU setting in jib script
- add an IP address to the bridge interface
- configure host pf to NAT and allow bridge traffic
- set defaultrouter and nameserver in the host

9.1 Change MTU in the jib script

GCP uses `vtnet` with MTU 1460, which jib fails on.

Apply the below patch to set the correct MTU. You may need to `cp /usr/share/examples/jails/jib /usr/local/bin/` first.

`patch /usr/local/bin/jib jib.patch`

```
--- /usr/local/bin/jib        2022-07-31 03:27:04.163245000 +0000
+++ jib.fixed 2022-07-31 03:41:16.710401000 +0000
@@ -299,14 +299,14 @@

               # Make sure the interface has been bridged
               if ! ifconfig "$iface$bridge" > /dev/null 2>&1; then
-                      new=$( ifconfig bridge create ) || return
+                      new=$( ifconfig bridge create mtu 1460 ) || return
                       ifconfig $new addm $iface || return
                       ifconfig $new name "$iface$bridge" || return
                       ifconfig "$iface$bridge" up || return
               fi

               # Create a new interface to the bridge
-              new=$( ifconfig epair create ) || return
+              new=$( ifconfig epair create mtu 1460 ) || return
               ifconfig "$iface$bridge" addm $new || return

               # Rename the new interface
```

9.2 Configure bridge interface

Configure the bridge interface in /etc/rc.conf so it is available in the firewall rules.

```
sysrc cloned_interfaces="bridge0"
sysrc ifconfig_bridge0="inet 192.168.1.1/24 mtu 1460 addm vtnet0 name vtnet0bridge up"
sysrc gateway_enable="yes"
sysrc pf_enable="yes"
```

9.3 Configure host pf

This basic /etc/pf.conf allow incoming packets on the bridge interface, and NATs them through the external interface:

```
ext_if="vtnet0"
bridge_if="vtnet0bridge"

set skip on lo
scrub in

# permissive NAT allows jail bridge and wireguard tunnels
nat on $ext_if inet from !($ext_if) -> ($ext_if:0)

block in
pass out

pass in proto tcp to port {22}
pass in proto icmp icmp-type { echoreq }
pass in on $bridge_if
```

Restart the host and make sure everything comes up correctly. You should see the following ifconfig:

```
vtnet0bridge: flags=8843<UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST> metric 0 mtu 1460
      ether 58:9c:fc:10:ff:90
      inet 192.168.1.1 netmask 0xffffff00 broadcast 192.168.1.255
      id 00:00:00:00:00:00 priority 32768 hellotime 2 fwddelay 15
      maxage 20 holdcnt 6 proto rstp maxaddr 2000 timeout 1200
      root id 00:00:00:00:00:00 priority 32768 ifcost 0 port 0
      member: vtnet0 flags=143<LEARNING,DISCOVER,AUTOEDGE,AUTOPTP>
              ifmaxaddr 0 port 1 priority 128 path cost 2000
      groups: bridge
```

9.4 Configure router and resolver for new jails

Set the default network gateway for new jails as described in the Networking chapter, and configure a default resolver.

```
sysrc -f /usr/local/etc/bastille/bastille.conf bastille_network_gateway="192.168.1.1"
echo "nameserver 8.8.8.8" > /usr/local/etc/bastille/resolv.conf
sysrc -f /usr/local/etc/bastille/bastille.conf bastille_resolv_conf="/usr/local/etc/
↪bastille/resolv.conf"
```

You can now create a VNET jail with `bastille create -V myjail 13.2-RELEASE 192.168.1.50/24 vtnet0`

CHAPTER

TEN

UPGRADING

This document outlines updating and upgrading jails hosted by Bastille.

Bastille can "bootstrap" multiple versions of FreeBSD to be used by jails. All jails do not NEED to be the same version (even if they often are), the only requirement here is that the "bootstrapped" versions are less than or equal to the host version of FreeBSD.

To keep releases updated, use `bastille update RELEASE`

To keep thick jails updated, use `bastille update TARGET`

10.1 Minor Release Upgrades - Legacy

To upgrade Bastille jails for a minor release (ie; 13.1 > 13.2) you can do the following:

10.1.1 Thick Jails

1. Use `bastille upgrade TARGET 13.2-RELEASE` to upgrade the jail to 13.2-RELEASE
2. Use `bastille upgrade TARGET install` to apply the updates
3. Reboot the jail `bastille restart TARGET`
4. Use `bastille upgrade TARGET install` to finish applying the upgrade
5. Upgrade complete!

10.1.2 Thin Jails

1. Ensure the new release version is bootstrapped: `bastille bootstrap 13.2-RELEASE`
2. Update the release (optional): `bastille update 13.2-RELEASE`
3. Stop the jail(s) that need to be updated.
4. Use `bastille upgrade TARGET 13.2-RELEASE` to automatically change the mount points to 13.2-RELEASE
5. Start the jail(s)
6. Upgrade complete!

10.2 Major Release Upgrades - Legacy

To upgrade Bastille jails for a major release (ie; 12.4 > 13.2) you can do the following:

10.2.1 Thick Jails

1. Use `bastille upgrade TARGET 13.2-RELEASE` to upgrade the jail to 13.2-RELEASE
2. Use `bastille upgrade TARGET install` to apply the updates
3. Reboot the jail `bastille restart TARGET`
4. Use `bastille upgrade TARGET install` to finish applying the upgrade
5. Force the reinstallation or upgrade of all installed packages (ABI change): `pkg upgrade -f` within each jail (or `bastille pkg ALL upgrade -f`)
6. Upgrade complete!

10.2.2 Thin Jails

1. Ensure the new release version is bootstrapped: `bastille bootstrap 13.2-RELEASE`
2. Update the release: `bastille update 13.2-RELEASE`
3. Stop the jail(s) that need to be updated.
4. Use `bastille upgrade TARGET 13.2-RELEASE` to automatically change the mount points to 13.2-RELEASE
5. Use `bastille etcupdate bootstrap 13.2-RELEASE` to bootstrap src for 13.2-RELEASE
6. Use `bastille etcupdate TARGET update 13.2-RELEASE` to update the contents of /etc for 13.2-RELEASE
7. Use `bastille etcupdate TARGET resolve` to resolve any conflicts
8. Start the jail(s)
9. Force the reinstallation or upgrade of all installed packages (ABI change): `pkg upgrade -f` within each jail (or `bastille pkg ALL upgrade -f`)
10. Upgrade complete!

10.3 Minor Release Upgrades - Pkgbase

To upgrade Bastille jails for a minor release (ie; 15.1 > 15.2) you can do the following:

10.3.1 Thick Jails

1. Use `bastille upgrade TARGET 15.2-RELEASE` to upgrade the jail to 15.2-RELEASE
2. Reboot the jail `bastille restart TARGET`
3. Upgrade complete!

10.3.2 Thin Jails

1. Ensure the new release version is bootstrapped: `bastille bootstrap --pkgbase 15.2-RELEASE`
2. Update the release (optional): `bastille update 15.2-RELEASE`
3. Stop the jail(s) that need to be updated.
4. Use `bastille upgrade TARGET 15.2-RELEASE` to automatically change the mount points to 15.2-RELEASE
5. Start the jail(s)
6. Upgrade complete!

10.4 Major Release Upgrades - Pkgbase

To upgrade Bastille jails for a major release (ie; 15.5 > 16.0) you can do the following:

10.4.1 Thick Jails

1. Use `bastille upgrade TARGET 16.0-RELEASE` to upgrade the jail to 16.0-RELEASE
2. Reboot the jail `bastille restart TARGET`
3. Force the reinstallation or upgrade of all installed packages (ABI change): `pkg upgrade -f` within each jail (or `bastille pkg ALL upgrade -f`)
4. Upgrade complete!

10.4.2 Thin Jails

1. Ensure the new release version is bootstrapped: `bastille bootstrap 16.0-RELEASE`
2. Update the release: `bastille update 16.0-RELEASE`
3. Stop the jail(s) that need to be updated.
4. Use `bastille upgrade TARGET 16.0-RELEASE` to automatically change the mount points to 16.0-RELEASE
5. Use `bastille etcupdate bootstrap 16.0-RELEASE` to bootstrap src for 16.0-RELEASE
6. Use `bastille etcupdate TARGET update 16.0-RELEASE` to update the contents of /etc for 16.0-RELEASE
7. Use `bastille etcupdate TARGET resolve` to resolve any conflicts
8. Start the jail(s)
9. Force the reinstallation or upgrade of all installed packages (ABI change): `pkg upgrade -f` within each jail (or `bastille pkg ALL upgrade -f`)
10. Upgrade complete!

10.5 Updating

To keep jails updated with the latest security patches and base, use the `bastille update` command.

10.5.1 Thick Jails

Use `bastille update TARGET` to update the jail with the latest patches and security updates.

10.5.2 Thin Jails

Use `bastille update RELEASE` to update the release that any thin jails are based on with the latest patches and security updates.

10.6 Revert Upgrade / Downgrade Process

The downgrade process (not usually needed) is similar to the upgrade process, only in reverse.

10.6.1 Thick Jails

Thick jails should not be downgraded and is not supported in general on FreeBSD.

10.6.2 Thin Jails

Not recommended, but you can run `bastille upgrade TARGET 13.1-RELEASE` to downgrade a thin jail. Make sure to run `bastille etcupdate TARGET update 13.1-RELEASE` to keep the contents of /etc updated with each release.

The pkg re-installation will also need to be repeated after the jail restarts on the previous release.

10.7 Old Releases

After upgrading all jails from one release to the next you may find that you now have bootstrapped a release that is no longer used. Once you've decided that you no longer need the option to revert the change you can destroy the old release.

`bastille list releases` to list all bootstrapped releases.

`bastille destroy X.Y-RELEASE` to fully delete the release, including the cache (cache is not used with pkgbase).

`bastille destroy -c|--no-cache X.Y-RELEASE` to retain the cache directory (not supported when using pkgbase).

CHAPTER

ELEVEN

MIGRATION

11.1 Bastille

Bastille supports migrations to a remote system using the `migrate` subcommand.

11.1.1 Prerequisites

There are a couple of things that need to be in place before running the `migrate` command.

First, you must have bastille configured both locally and remotely to use the same filesystem configuration. ZFS on both, or UFS on both.

Second, you must create a user on the remote system that will be used to migrate the jail. The user must be able to log in via SSH using either key-based authentication, or password based authentication. The user also needs `sudo` permissions on the remote system. This user should then be given as the `USER` arg in the `migrate` command.

If you don't want to use `sudo`, we support using `doas` as the super-user command. Simply set `--doas` as one of the options when running the `migrate` command.

If you are using key-based auth, the keys should be stored in the default location at `$HOME/.ssh/id_rsa`, where `$HOME` is the users home directory. This is the default location for ssh keys, and where Bastille will try to load them from.

If you want to use password based authentication, simply run `bastille migrate -p TARGET USER@HOST`. This will prompt you to enter the password for the remote system, which Bastille will then use during the migration process.

11.1.2 Migration

To migrate a jail (or multiple jails) we can simply run `bastille migrate TARGET USER@HOST`. This will export the jail(s), send them to the remote system, and import them.

The `migrate` sub-command includes the `-a|--auto` option, which will auto-stop the old jail, migrate it, and attempt to start the migrated jail on the remote system after importing it. See the warning below about auto-starting the migrated jail.

WARNING: Every system is unique, has different interfaces, bridges, and network configurations. It is possible, with the right configuration, for jails to start and work normally. But for some systems, it will be necessary to edit the `jail.conf` file of the migrated jail to get it working properly.

Using `-l|--live` mode (ZFS only) will leave the local jail running, and the remote jail stopped.

Using `-a|--auto` mode will stop the local jail, and start the remote jail.

Using the `-l|--live` and `-a|--auto` options together will migrate the jail while it is running, then stop the local jail, and start the remote jail.

You can optionally set `-d|--destroy` to have Bastille destroy the old jail on completion.

You can also set `-b|--backup` to retain the archives remotely. They will be copied into `${bastille_backupsdir}`.

11.2 iocage

Stop the running jail and export it:

```
iocage stop jailname
iocage export jailname
```

Move the backup files (.zip and .sha256) into Bastille backup dir (default: /usr/local/bastille/backups/):

```
mv /iocage/images/jailname_$(date +%F).* /usr/local/bastille/backups/
```

for remote systems you can use rsync:

```
rsync -avh /iocage/images/jailname_$(date +%F).* root@10.0.1.10:/usr/local/bastille/
↪backups/
```

Import the iocage backup file (use zip file name)

```
bastille import jailname_$(date +%F).zip
```

Bastille will attempt to configure your interface and IP from the `config.json` file, but if you have issues you can configure it manually.

```
bastille edit jailname
ip4.addr = bastille0|192.168.0.1/24;
```

You can use your primary network interface instead of the virtual `bastille0` interface as well if you know what you're doing.

CHAPTER

TWELVE

CENTRALIZED ASSETS

Sometimes it is preferable to share applications, libraries, packages or even directories and files across multiple jails.

Or perhaps we just want to avoid all the time it takes to create a jail, and manually configure it with the packages we normally use.

Bastille offers a number of ways to do the above.

12.1 Templates

A template is a predefined file containing instructions to execute on a targeted jail. This is one of the easiest ways to create a repeatable environment for your Bastille jails. Simply create your template, the execute it on as many jails as you prefer.

```
ishmael ~ # bastille template "jail1 jail2" project/template
```

See *Templates* for more details on templates.

12.2 Mounting

On of the fastest ways to share directories and files across multiple jails is with the `bastille mount` command.

The following command will mount `/my/host/directory` into `jail1` and `jail2` at `/my/jail/directory` with read and write access. To mount with read only access, simply use `ro` instead of `rw` as the option.

```
ishmael ~ # bastille mount "jail1 jail2" /my/host/directory /my/jail/directory nullfs rw␣
↪0 0
```

12.3 Cloning

Bastille allows you to create duplicate of your jail using `bastille clone`. To clone your jail, use the following command.

```
ishmael ~ # bastille clone myjail mynewjail 10.0.0.3
```

This will create an exact duplicate of `myjail` at `mynewjail`.

12.4 Custom Releases

Bastille allows creating custom releases from jails, then using those releases to create more jails.

To start, we must first create our jail. Make sure it is a thick jail, as this process will not work with any other jail types.

```
ishmael ~ # bastille create -T myjail 14.2-RELEASE 10.0.0.1
```

Once the jail is up and running, configure it to your liking, then run the following commmand to create a custom release based on your jail.

```
ishmael ~ # bastille convert myjail myrelease
```

Once this process completes, you will be able to run the following command to create a jail based off your newly created release.

Please note that using this approach is experimental. It will be up to the end user to keep track of which official FreeBSD release their custom release is based on. The `osrelease` config variable will be set to your custom release name inside the `jail.conf` file.

```
ishmael ~ # bastille create -T --no-validate myjail myrelease 10.0.0.2
```

CHAPTER

THIRTEEN

TEMPLATES

Looking for ready made CI/CD validated Bastille Templates?

Bastille features a template system, allowing you to automate just about anything from executing arbitrary commands to copying files all with a simple file called a Bastillefile. A template is applied by running `bastille template TARGET project/template` and can also be applied to multiple targets in one go.

Before we dive into creating templates, lets take a look at the supported hooks, as well as a brief overview of what each one is capable of.

13.1 Template Hooks

The following table shows a list of supported template hooks, their format, and one example of how you might use each one.

HOOK	format	example
ARG[+]	ARG=VALUE	MINECRAFT_MEMX="1024M"
CMD	/bin/sh command	/usr/bin/chsh -s /usr/local/bin/zsh
CONFIG	set property value	set allow.mlock 1
CP	path(s)	etc root usr
INCLUDE	template path/URL	http?://TEMPLATE_URL or project/path
LIMITS	resource value	memoryuse 1G
LINE_IN_FILE	line path	word /usr/local/word/word.conf
MOUNT	fstab syntax	/host/path /jail/path nullfs ro 0 0
[H]PKG	port/pkg name(s)	vim-console zsh git-lite tree htop
RDR	tcp port port	tcp 2200 22 (proto hostport jailport)
RENDER	/path/file.txt	/usr/local/etc/gitea/conf/app.ini
RESTART		
SERVICE	service command	'nginx start' OR 'postfix reload'
SYSRC	sysrc command(s)	nginx_enable=YES
TAGS	tag1 tag2 tag3	prod web

13.2 Template Hook Descriptions

`ARG` - set an ARG value to be used in the template

ARGS will default to the value set inside the template, but can be changed by including `--arg ARG=VALUE` when running the template.

Multiple ARGS can also be specified as seen below. If no ARG value is given, Bastille will show a warning, but continue on with the rest of the template.

```
ishmael ~ # bastille template azkaban sample/template --arg ARG=VALUE --arg ARG1=VALUE
```

The ARG hook has a wide range of functionality, including passing KEY=VALUE pairs to any templates called with the INCLUDE hook. See the following example…

```
ARG JAIL
ARG IP

INCLUDE other/template --arg JAIL=${JAIL} --arg IP=${IP}
```

If the above template is called with `--arg JAIL=myjail --arg IP=10.3.3.3`, these values will be passed along to `other/template` as well, with the matching variable. So `${JAIL}` will be `myjail` and `${IP}` will be `10.3.3.3`.

The ARG hook has three values that are built in, and will differ for every jail. The values are `JAIL_NAME`, `JAIL_IP`, and `JAIL_IP6`. These can be used inside any template without setting the values at the top of the Bastillefile. The values are automatically retrieved from the targeted jails configuration.

`ARG+` - the + makes the `ARG` mandatory

`CMD` - run the specified command

`CONFIG` - set the specified property and value

`CP` - copy specified files from template directory to specified path inside jail

The `CP` hook will recursively copy all of the specified directories from the `project/template` directory into the jail. If you have `CP usr etc` for example, it will recursively copy `project/template/usr` and `project/template/etc` into `/usr` and `/etc` of the jail directory.

So, if you have `project/template/usr/local/share/myapp.conf`, it will be copied into the jail, and placed at `/usr/local/share/myapp.conf`.

Note: due to the way FreeBSD segregates user-space, the majority of your overlayed template files will be in `/usr/local`. The few general exceptions are the `/etc/hosts`, `/etc/resolv.conf`, and `/etc/rc.conf.local`.

The above example of `usr` and `etc` will include anything under `usr` and `etc` inside the template. You do not need to list individual files. Just include the top-level directory name. List these top-level directories one per line.

`INCLUDE` - specify a template to include. Make sure the template is bootstrapped, or you are using the template url

`LIMITS` - set the specified resource value for the jail

`LINE_IN_FILE` - add specified word to specified file if not present

`MOUNT` - mount specified files/directories inside the jail

`PKG` - install specified packages inside jail

`HPKG` - install specified packages inside jail using the host pkg

`RDR` - redirect specified ports to the jail

There are two versions of the `RDR` hook:

- Simple: proto hostport jailport, as shown in the table above
- Advanced: [ipv4 ip46 dual] interface source-ip dest-ip proto hostport jailport

An example of the advanced `RDR`:

```
RDR ipv4 vtnet0 192.168.0.1 any tcp 2022 22
```

This forwards port 22 in the jail to port 2022 on the host, allowing only connections from 192.168.0.1, an IP address external to the host, all other IPs will be denied.

Note that `dual` can only be used if both source-ip and dest-ip are `any`.

`RENDER` - replace ARG values inside specified files inside the jail

If a directory is specified here, ARGS will be replaced in all files underneath, or recursively.

`RESTART` - restart the jail

`SERVICE` - run *service* command inside the jail with specified arguments

`SYSRC` - run *sysrc* inside the jail with specified arguments

`TAGS` - adds specified tags to the jail

Pro Tip: Most Bastille commands can be placed inside the Bastillefile. But only the above listed hooks are tested and supported officially. It is also possible to formulate any regular Bastille command to be run by the template. The following example will clarify…

```
RDR reset
NETWORK add vtnet1 DHCP
```

The above snippet, when included in a template will essentially run `bastille rdr TARGET reset` and `bastille network TARGET add vtnet1 DHCP` inside the jail respectively. Although not fully tested and documented, they should still work as expected.

13.3 Special Hook Cases

`ARG` will always treat an ampersand "``&``" literally, without the need to escape it. Escaping it will cause errors.

13.4 Passing ARG Values From a File

When a template declares a lot of `ARG` values, listing them all on the command line with repeated `--arg NAME=VALUE` flags becomes unwieldy. The `template` sub-command accepts a `--arg-file` option that points at a plain-text file containing the values.

```
ishmael ~ # bastille template TARGET project/template --arg-file /path/to/args.env
```

The file must already exist; Bastille will exit with `[ERROR]: File not found: <path>` otherwise.

13.4.1 File Format

Each line in the file is a `NAME=VALUE` pair, one `ARG` per line. The `NAME` portion must start at the beginning of the line (Bastille looks up each `ARG` with an anchored match on `^NAME=`), and `VALUE` is everything that follows the first `=`.

```
# /path/to/args.env
MINECRAFT_MEMX=2048M
MINECRAFT_MEMS=2048M
```

Lines that do not begin with `NAME=` (for example blank lines or shell-style comments) are ignored because they cannot match the anchored lookup, so they are safe to include for readability. An `ARG` whose name does not appear in the file simply falls through to its default.

13.4.2 Precedence

For any given `ARG NAME` referenced inside the Bastillefile, the value is resolved in this order:

1. `--arg NAME=VALUE` passed on the command line (highest priority).
2. A `^NAME=` line found in the `--arg-file`.
3. The default value supplied next to the `ARG` declaration in the Bastillefile (lowest priority).

This means you can keep a baseline of values in a file and selectively override any of them on the command line without editing the file:

```
ishmael ~ # bastille template azkaban games/minecraft-server \
    --arg-file /etc/bastille/minecraft.env \
    --arg MINECRAFT_MEMX=4096M
```

In the example above every `ARG` is sourced from `minecraft.env` except `MINECRAFT_MEMX`, which is taken from the explicit `--arg` flag.

13.4.3 Notes

- Only the first `--arg-file` on the command line is honored; subsequent occurrences are ignored.
- The path is read on each `ARG` lookup, so the file must remain readable for the duration of the template run.
- Values are passed through the same escaping logic as `--arg`, so the rule from *Special Hook Cases* applies — an ampersand "``&``" in a value is treated literally and must not be escaped.

13.5 Bootstrapping Templates

The official templates for Bastille are all on Gthub, and mirror the directory structure of the ports tree. So, `nginx` is in the `www` directory in the templates repo, just like it is in the FreeBSD ports tree. To bootstrap the entire set of official templates, run the following command:

```
bastille bootstrap https://github.com/bastillebsd/templates
```

This will bootstrap all official templates into the templates directory at `/usr/local/bastille/templates`. You can then use the `bastille template` command to apply any of the templates.

```
bastille template TARGET www/nginx
```

13.6 Creating Templates

Templates should be created and placed inside the templates directory in the `project/template` format. Alternatively you can run the `bastille template` command from a relative path, making sure it is still in the above `project/template` format.

Place any uppercase template hook into `project/template/Bastillefile` in any order to automate jail setup as needed.

Any files included in the `project/template` directory can be copied into the jail using the `CP` hook. For example, if I have `project/template/usr/local/etc/custom.conf` I can use the following template to copy the entire contents of `usr` into my jail. Bastille will not overwrite `/usr` inside the jail. It only copies the files in.

```
CP usr /
```

See Bastille Templates for examples to get started on writing your own templates.

13.7 Applying Templates

To apply a template to a jail, run the following command.

```
ishmael ~ # bastille template ALL project/template
[proxy01]:
Copying files...
Copy complete.
Installing packages.
pkg already bootstrapped at /usr/local/sbin/pkg
vulnxml file up-to-date
0 problem(s) in the installed packages found.
Updating bastillebsd.org repository catalogue...
[cdn] Fetching meta.txz: 100%    560 B   0.6kB/s    00:01
[cdn] Fetching packagesite.txz: 100%  121 KiB 124.3kB/s    00:01
Processing entries: 100%
bastillebsd.org repository update completed. 499 packages processed.
All repositories are up to date.
Checking integrity... done (0 conflicting)
The most recent version of packages are already installed
Updating services.
cron_flags: -J 60 -> -J 60
sendmail_enable: NONE -> NONE
syslogd_flags: -ss -> -ss
Executing final command(s).
chsh: user information updated
Template Complete.

[web01]:
Copying files...
Copy complete.
Installing packages.
pkg already bootstrapped at /usr/local/sbin/pkg
vulnxml file up-to-date
0 problem(s) in the installed packages found.
Updating pkg.bastillebsd.org repository catalogue...
[poudriere] Fetching meta.txz: 100%    560 B   0.6kB/s    00:01
[poudriere] Fetching packagesite.txz: 100%  121 KiB 124.3kB/s    00:01
Processing entries: 100%
pkg.bastillebsd.org repository update completed. 499 packages processed.
Updating bastillebsd.org repository catalogue...
[poudriere] Fetching meta.txz: 100%    560 B   0.6kB/s    00:01
[poudriere] Fetching packagesite.txz: 100%  121 KiB 124.3kB/s    00:01
Processing entries: 100%
bastillebsd.org repository update completed. 499 packages processed.
All repositories are up to date.
Checking integrity... done (0 conflicting)
The most recent version of packages are already installed
Updating services.
cron_flags: -J 60 -> -J 60
sendmail_enable: NONE -> NONE
syslogd_flags: -ss -> -ss
Executing final command(s).
```

(continues on next page)

(continued from previous page)

```
chsh: user information updated
Template Complete.
```

Notice that if we choose `ALL` as the target, the template is applied to all jails. See *Targeting* for more details on targeting jails.

13.8 Using Ports in Templates

Sometimes when creating a template, we need special options for a package, or a newer version than pkg offers. The solution for such cases, or a case like `minecraft-server` which has NO compiled option, is to use ports. A working example of this is the `minecraft-server` template in the template repo. The main lines needed to use this is first to mount the ports directory, then compile the port. Below is an example of the `minecraft-server` template where this was used.

```
ARG MINECRAFT_MEMX="1024M"
ARG MINECRAFT_MEMS="1024M"
ARG MINECRAFT_ARGS=""
CONFIG set enforce_statfs=1;
CONFIG set allow.mount.fdescfs;
CONFIG set allow.mount.procfs;
RESTART
PKG dialog4ports tmux openjdk17
MOUNT /usr/ports usr/ports nullfs ro 0 0
CP etc /
CP var /
CMD make -C /usr/ports/games/minecraft-server install clean
CP usr /
SYSRC minecraft_enable=YES
SYSRC minecraft_memx=${MINECRAFT_MEMX}
SYSRC minecraft_mems=${MINECRAFT_MEMS}
SYSRC minecraft_args=${MINECRAFT_ARGS}
SERVICE minecraft restart
RDR tcp 25565 25565
```

The `MOUNT` line mounts the ports directory, then the `CMD` make line makes the port. This can be modified to use any port in the ports tree.

CHAPTER

FOURTEEN

HARDENEDBSD

Bastille supports HardenedBSD as an OS since it is FreeBSD based. There are some differences in how HBSD handles release names, updates, and upgrades.

Most of the Bastille commands will work with HardenedBSD, but please report any bugs you may find.

There are a number of ways in which HardenedBSD differs from FreeBSD. Most of the functionality is the same, but some things are different. See the following examples…

14.1 Bootstrap

HardenedBSD follows the `STABLE` branches of FreeBSD, and releases are named `X-stable`, where `X` is the major version of a given FreeBSD branch/release.

It also has a `current` release, which follows the master/current branch for the latest FreeBSD release.

When bootstrapping a release, use the above release keywords.

14.2 Updating

To update HardenedBSD jails/releases you can do the following:

14.2.1 Thick Jails

1. Use `bastille update TARGET` to update the jail
2. Upgrade complete!

14.2.2 Thin Jails

See `bastille update RELEASE` to update thin jails, as thin jails are based on a given release.

14.2.3 Releases

1. Use `bastille update 15-stable` to update the release to the latest version
2. Update complete!

14.3 Upgrading

To upgrade HardenedBSD jails to a different (higher) release (ie; 14-stable > 15-stable) you can do the following:

14.3.1 Thick Jails

1. Use `bastille upgrade TARGET current` to upgrade the jail to the `current` release
2. Force the reinstallation or upgrade of all installed packages (ABI change): `pkg upgrade -f` within each jail (or `bastille pkg ALL upgrade -f`)
3. Upgrade complete!

14.3.2 Thin Jails

1. Ensure the new release is bootstrapped: `bastille bootstrap 15-stable`
2. Update the release: `bastille update 15-stable`
3. Stop the jail(s) that need to be updated.
4. Use `bastille upgrade TARGET 15-stable` to automatically change the mount points to 15-stable
5. Start the jail(s)
6. Force the reinstallation or upgrade of all installed packages (ABI change): `pkg upgrade -f` within each jail (or `bastille pkg ALL upgrade -f`)
7. Upgrade complete!

14.3.3 Releases

The `upgrade` sub-command does not support upgrading a release to a different release. See `bastille bootstrap` to bootstrap the new release.

14.4 Limitations

Bastille tries its best to determine which `*BSD` you are using. It is possible to mix and match any of the supported BSD distributions, but it is up to the end user to ensure the correct environment/tools when doing so. See below…

- Running HardenedBSD jails/releases requires many of the tools found only in the HardenedBSD base.
- Running FreeBSD jails/releases requires many of the tools found only in the FreeBSD base.

CHAPTER

FIFTEEN

LINUX JAILS

Bastille can create Linux jails using the `debootstrap` tool. When attempting to create a Linux jail, Bastille will need to load some modules as well as install the `debootstrap` package.

15.1 Getting Started

To get started, run `bastille setup linux` to load required modules and install the `debootstrap` package.

15.2 Bootstrapping a Linux Release

To bootstrap a Linux release, run `bastille bootstrap bionic` or whichever release you want to bootstrap. Once bootstrapped, we can use the `--linux|-L` option to create a Linux jail.

15.3 Creating a Linux Jail

To create a Linux jail, run `bastille create -L mylinuxjail bionic 10.1.1.3`. This will create and initialize your jail using the `debootstrap` tool.

Once the jail is created, proceed to do your "linux stuff".

15.4 Limitations

- Linux jails are still considered experimental.
- Linux jails cannot be created with any type of VNET options.

CHAPTER

SIXTEEN

PKGBASE

Pkgbase is the new method for managing the base system on a FreeBSD host or jail. It is considered experimental for 15.0-RELEASE, but will be made the default for version 16.0-RELEASE and above.

16.1 Bootstrap

To bootstrap a release using pkgbase, run `bastille bootstrap --pkgbase RELEASE`. For version 14, it is not supported. For version 15 it is optional, but for version 16 and above, it is the default method of bootstrapping a release.

To customize the 'pkgbase package set' used for bootstrapping, change the 'bastille_pkgbase_packages' setting located in `/usr/local/etc/bastille/bastille.conf`. See also *Configuration*.

16.2 Update

To update a release created with pkgbase, simply run `bastille update RELEASE` as you would with legacy releases.

To update a thick jail, run `bastille update TARGET` as you would with legacy releases.

To update a thin jail, you must update the release that it is based on.

16.3 Upgrade

Upgrading is not supported for releases. See `bastille bootstrap RELEASE` to bootstrap the required release.

Upgrading is supported for both thin and thick jails. Thin jails will have their mount points adjusted, and you will need to run `bastille etcupdate` on them when upgrading from a major release to a newer major release. For example, 15.0-RELEASE to 16.0-RELEASE.

16.4 Converting to Pkgbase

Thick jails that are running legacy releases will have to be converted to pkgbase before attempting to upgrade to 16.0-RELEASE. This can be done in two ways.

1. Enter the jail, fetch the `pkgbasify` script, and run it.

```
fetch https://github.com/FreeBSDFoundation/pkgbasify/raw/refs/heads/main/pkgbasify.lua
chmod +x pkgbasify.lua
./pkgbasify.lua
```

2. Fetch the `pkgbasify` script and run it from the host using the `--jail TARGET` option.

```
fetch https://github.com/FreeBSDFoundation/pkgbasify/raw/refs/heads/main/pkgbasify.lua
chmod +x pkgbasify.lua
./pkgbasify.lua --jail TARGET
```

Converting a release to pkgbase can be done the same way, but we recommend simply destroying and re-bootstrapping it using pkgbase. This will not work if you are running thin jails based on the release in question. In such a case, follow step 2 above.

CHAPTER

SEVENTEEN

ZFS SUPPORT

Bastille 0.4 added initial support for ZFS. `bastille bootstrap` and `bastille create` will generate ZFS volumes based on settings found in the `bastille.conf`. This section outlines how to enable and configure Bastille for ZFS. As of Bastille 0.13 you no longer need to do these steps manually. The setup program when you run:

```
bastille setup
```

will create the zfs settings for you IF you are running zfs. This section is left in the documents for historical purposes, and so you can understand what the setup program is doing AND so if you need to tweak your settings for some reason.

Two values are required for Bastille to use ZFS. The default values in the `bastille.conf` are NO and empty. Populate these two to enable ZFS.

```
## ZFS options
bastille_zfs_enable=""                                  ## default: "NO"
bastille_zfs_zpool=""                                   ## default: ""
bastille_zfs_prefix="bastille"                          ## default: "${bastille_zfs_
↪zpool}/bastille"
bastille_zfs_options="-o compress=lz4 -o atime=off"     ## default: "-o compress=lz4 -o␣
↪atime=off"
```

Example

```
ishmael ~ # sysrc -f /usr/local/etc/bastille/bastille.conf bastille_zfs_enable=YES
ishmael ~ # sysrc -f /usr/local/etc/bastille/bastille.conf bastille_zfs_zpool=ZPOOL_NAME
```

Replace `ZPOOL_NAME` with the zpool you want Bastille to use. Tip: `zpool list` and `zpool status` will help. If you get 'no pools available' you are likely not using ZFS and can safely ignore these settings.

By default, bastille will use `ZPOOL_NAME/bastille` as its working zfs dataset. If you want it to use a specific dataset on your pool, set `bastille_zfs_prefix` to the dataset you want bastille to use. DO NOT include the pool name.

Example

```
ishmael ~ # sysrc -f /usr/local/etc/bastille/bastille.conf bastille_zfs_prefix=apps/
↪bastille
```

The above example will set `ZPOOL_NAME/apps/bastille` as the working zfs dataset for bastille.

Bastille will mount the datasets it creates at `bastille_prefix` which defaults to `/usr/local/bastille` If this is not desirable, you can change it at the top of the config file.

17.1 Altroot

If a ZFS pool has been imported using `-R` (altroot), your system will automatically add whatever the `altroot` is to any `zfs mount` commands. Bastille supports using an `altroot`, and there should be no issues using this feature.

One thing to note though, is that you MUST NOT include your `altroot` path in the `bastille_prefix`. For example, if you imported your pool with `zpool import -R /mnt poolname`, and you wish for your jails to live at `/mnt/poolname/bastille` then `bastille_prefix` should be set to `/poolname/bastille` without the `/mnt` part.

If you do accidentally add the `/mnt` part, your datasets will be mounted at `/mnt/mnt/poolname/bastille` and Bastille will throw all kinds of errors due to not finding the proper paths.

17.2 Jailing a Dataset

It is possible to "jail" a dataset. This means mounting a datset into a jail, and being able to fully manage it from within the jail.

To add a dataset to a jail, we can run `bastille zfs TARGET jail pool/dataset /path/inside/jail`. This will assign `pool/dataset` to the jail and mount it at `/path/inside/jail`.

You can manually change the path where the dataset will be mounted by `bastille edit TARGET zfs.conf` and adjusting the path after you have added it, bearing in mind the warning below.

WARNING: Adding or removing datasets to the `zfs.conf` file can result in permission errors with your jail. It is important that the jail is first stopped before attempting to manually configure this file. The format inside the file is simple.

```
pool/dataset /path/in/jail
pool/other/dataset /other/path/in/jail
```

To remove a dataset from being jailed, we can run `bastille zfs TARGET unjail pool/dataset`.

NOTE: You must unjail any jailed datasets before attempting to destroy a jail.

CHAPTER

EIGHTEEN

COPYRIGHT

Note: this documentation is included with the source code in `docs`.